

Duale Hochschule Baden-Württemberg Mosbach

Multidimensionale Evaluation von LLM-basierten Coding-Agenten jenseits funktionaler Testmetriken

Verfasser:

Chris David Kaufmann, Luca di Siro, Daniel Ertel, Richard Beser

Studiengang Wirtschaftsinformatik

Profil: Angewandte Künstliche Intelligenz

Bearbeitungszeitraum: 6 Semester

Matrikelnummer:	7940617, 5921556, 7846151, 3168151
Kurs:	MOS-WWI23A
E-Mail:	chr.kaufmann.23@lehre.mosbach.dhbw.de luc.disiro.23@lehre.mosbach.dhbw.de dan.ertel.23@lehre.mosbach.dhbw.de ric.beser@lehre.mosbach.dhbw.de
Studiengangsleiter:	Dirk Palleduhn
Betreuender Dozent:	Dr. Schalles
E-Mail:	c.schalles@lehre.mosbach.dhbw.de

Inhaltsverzeichnis

1. Einleitung	3
1.1 Ausgangslage und Problemstellung	3
1.2 Zielsetzung und Relevanz	3
1.3 Forschungsfragen und Hypothesen	4
2. Technologische Fundierung und Begründung des Studiendesigns	6
2.1 Wahl der Programmiersprache und der Analyse-Infrastruktur	6
2.2 Wahl der Large Language Models und provider-spezifischer Architekturen	6
2.3 Operationalisierung von Agentenmodus und Prompt-Qualität	7
3. Methodisches Vorgehen im Detail: Qualitätsbewertungsmethoden	9
3.1 Source Lines of Code (SLOC) und Lines of Code (LOC)	9
3.2 Zyklomatische Komplexität nach McCabe	9
3.3 Maintainability Index (Wartbarkeitsindex)	10
3.4 Hallucination Flags und LLM-spezifische Heuristiken	10
3.5 Aggregierter qualitativer Gesamtscore	11
4. Statistische Methodik	13
4.1 Friedman-Test	13
4.2 Cochran-Q-Test	13
4.3 Wilcoxon-Signed-Rank-Test	14
4.4 McNemar-Test	14
4.5 Holm-Bonferroni-Korrektur	15
4.6 OLS-Regression zur Prüfung von H1	15
4.7 Interpretationsrahmen	16
5. Methodik zur Normalisierung des Tokenverbrauchs	17
5.1 Allgemeine Normalisierungsformeln und Kennzahlen	17
5.2 Normalisierung der Token-Metriken für Anthropic Claude	18
5.3 Normalisierung der Token-Metriken für OpenAI Codex	18
5.4 Normalisierung der Token-Metriken für Google Gemini CLI	19
6. Auswertung der Ergebnisse	20
6.1 Operative Auswertung im fairen Bereich $n = 70$	20
6.2 Operative Vollauswertung $n = 96$	21
6.3 Artefaktqualität im fairen Vergleich $n = 70$	22
6.4 Post-hoc-Analyse der Artefaktqualität	24
6.5 Auswertung des Agentenmodus: Direct vs. Self-Refining	27
6.6 Auswertung der Prompt-Qualität: Low, Medium und High	28
6.7 Ökonomische Token-Effizienz und abnehmender Grenznutzen	30
6.8 Fazit und Synthese der Ergebnisse	32
7. Zusammenfassung, Ausblick und Bewertung	34
7.1 Zusammenfassung	34
7.2 Ausblick	35
7.3 Kritische Bewertung	35

1. Einleitung

1.1 Ausgangslage und Problemstellung

Die Evaluation von LLM-basierter Codegenerierung erfolgt in etablierten Benchmarks häufig über funktionale Korrektheit. Chen et al. beschreiben für HumanEval, dass „correctness is defined by passing a set of unit tests“ (Chen et al. 2021, S. 9). Diese Form der Bewertung ist für die Prüfung grundlegender Funktionsfähigkeit geeignet, stößt jedoch bei leistungsstarken Modellen zunehmend an Grenzen. Akhtar et al. weisen darauf hin, dass bei konvergierenden Modelleleistungen „benchmarks lose discriminative power“ (Akhtar et al. 2026, S. 1). Für die vorliegende Untersuchung ist dies insbesondere relevant, da die initial getesteten Agenten-Konfigurationen bei einfachen algorithmischen Aufgaben nahezu durchgehend erfolgreich waren.

Daraus ergibt sich eine methodische Lücke: Eine bestandene Testsuite belegt zwar funktionale Lösbarkeit, erlaubt jedoch keine hinreichende Aussage über nicht-funktionale Qualitätsmerkmale des generierten Codes. Sun et al. formulieren diese Anforderung prägnant als Ziel, dass generierter Code „not only passes tests but truly passes with quality“ (Sun et al. 2026, S. 1,2). Ergänzend zeigen Liu et al., dass vorhandene Testfälle in Code-Benchmarks „limited in both quantity and quality“ sein können (Liu et al. 2023, S. 1). Vor diesem Hintergrund ist eine multidimensionale Evaluation erforderlich, die neben der sichtbaren Pass-Rate auch Wartbarkeit, Test-Robustheit, Constraint-Konformität, Laufzeit und Token-Effizienz berücksichtigt.

1.2 Zielsetzung und Relevanz

Das primäre Ziel dieser Arbeit besteht in der Konzeption, Implementierung und statistischen Absicherung eines multidimensionalen Evaluations-Frameworks für KI-Coding-Agenten. Durch die Erweiterung der bestehenden Pipeline um sekundäre Metriken soll eine differenzierte Bewertung ermöglicht werden, die sowohl funktionale als auch ressourcenbezogene Kriterien einbezieht. Von zentraler Bedeutung ist hierbei die Analyse der Token- und Ressourceneffizienz, da der Ressourcenverbrauch im produktiven Unternehmenseinsatz einen kritischen Skalierungsfaktor darstellt.

Da die verschiedenen API-Provider divergierende Abrechnungs- und Erfassungsmodelle für Eingabe-, Ausgabe-, Caching- und Reasoning-Tokens verwenden, bildet die Entwicklung einer mathematischen Normalisierungsmethodik einen wesentlichen Kern dieser Untersuchung. Nur durch eine solche Harmonisierung lässt sich eine valide Vergleichbarkeit der Token- und Ressourceneffizienz herstellen. Die Ergebnisse dieser Arbeit sollen somit eine fundierte Entscheidungsgrundlage für das Enterprise-Management liefern, indem sie aufzeigen, wie die Stellschrauben Modellwahl, Agentenarchitektur und Prompt-Qualität zur Optimierung der Ressourceneffizienz beitragen können.

1.3 Forschungsfragen und Hypothesen

Die wissenschaftliche Untersuchung widmet sich der Beantwortung mehrerer miteinander verknüpfter Forschungsfragen, die als formale Hypothesen (H1–H4) operationalisiert und im empirischen Teil geprüft werden.

Aus der oben skizzierten Problemstellung leiten sich vier Forschungsfragen ab, deren Beantwortung den empirischen Teil der Arbeit strukturiert:

F1 — Welchen marginalen Beitrag liefert zusätzlicher Tokenverbrauch zur funktionalen Leistung agentischer Coding-CLIs in einem fest umrissenen Aufgabenraum?

F2 — Verbessert ein Self-Refining-Modus mit automatisierten Korrekturschleifen die Robustheit gegenüber Randfällen gegenüber einer direkten Code-Generierung?

F3 — Wie wirkt sich eine systematische Erhöhung der Prompt-Qualität auf den Ressourcenverbrauch je Lauf aus?

F4 — Führt eine Erhöhung der Prompt-Qualität zu einem signifikant höheren qualitativen Gesamtscore bei gleichzeitig reduziertem Tokenverbrauch?

Diese Forschungsfragen werden im Folgenden als formale Hypothesen (H1–H4) operationalisiert und im empirischen Teil (Kapitel 6) geprüft.

Die erste Hypothese (H1) postuliert einen Sättigungseffekt des Tokenverbrauchs: Im untersuchten Aufgabenraum erklärt zusätzlicher Tokeneinsatz keine signifikant höhere funktionale Leistung. Diese Annahme leitet sich aus den empirischen Skalierungsgesetzen ab, nach denen „the loss scales as a power-law with model size, dataset size, and the amount of compute used for training“ (Kaplan et al. 2020, S. 1); für hinreichend leistungsstarke Modelle in einem fest umrissenen Aufgabenraum impliziert dieses Power-Law einen abnehmenden Grenznutzen, der sich operationalisieren lässt als nicht signifikanter Zusammenhang zwischen log-transformiertem Tokenverbrauch und Test-Pass-Rate in einem OLS-Modell. Die Hypothese gilt als bestätigt, wenn der Koeffizient für $\log(\text{Token})$ nicht signifikant von null verschieden ist und das Bestimmtheitsmaß R^2 nahe null liegt.

Die zweite Hypothese (H2) betrifft den Architektureffekt autonomer Agenten. Es wird vermutet, dass ein Self-Refining-Ansatz, der auf wiederholter Selbstbewertung und automatisierten Fehlerkorrekturschleifen basiert, die Robustheit gegenüber komplexen Randfällen – operationalisiert über die Hidden-Test-Pass-Rate – gegenüber einer direkten Code-Generierung signifikant verbessert. Diese Annahme wird dadurch gestützt, dass LLMs „do not always generate the best output on their first try“ (Madaan et al. 2023, S. 1) und Self-Refine darauf basiert, dass „the same LLM provides feedback for its output and uses it to refine itself, iteratively“ (Madaan et al. 2023, S. 1). Geprüft wird H2 über den Wilcoxon-Signed-Rank-Test der gepaarten Hidden-Pass-Raten in den Modi Direct und Self-Refining.

Die dritte Hypothese (H3) betrachtet den Prompteffekt aus ökonomischer Perspektive. Sie besagt, dass eine systematische Erhöhung der Prompt-Qualität den normalisierten

Tokenverbrauch je Lauf signifikant senkt, da präzisere Anweisungen den explorativen Anteil der Inferenz reduzieren. Prompt Engineering wird dabei als relevanter Einflussfaktor verstanden, da Prompts als „task-specific instructions, known as prompts, to enhance model efficacy“ beschrieben werden (Sahoo et al. 2025, S. 1) und „the selection and composition of prompt examples can significantly influence model behavior“ (Sahoo et al. 2025, S. 2). Geprüft wird H3 über den Friedman-Test der normalisierten Kontexttokens über die drei Prompt-Level Low, Medium und High in der fair gepaarten Artefaktbasis.

Die vierte Hypothese (H4) führt diese Aspekte in einer Synthese zusammen: Es wird angenommen, dass Expert-Prompts den qualitativen Gesamtscore gegenüber Low- und Medium-Prompts signifikant erhöhen und zugleich den Tokenverbrauch reduzieren. Diese Hypothese unterscheidet sich von H3 darin, dass sie nicht nur einen Tokenrückgang, sondern zusätzlich einen signifikanten Qualitätsanstieg verlangt. Sun et al. weisen darauf hin, dass bisherige Evaluationen häufig auf funktionale Korrektheit fokussieren, während nicht-funktionale Qualitätsmerkmale wie „performance efficiency, maintainability, and security“ ebenfalls berücksichtigt werden müssen (Sun et al. 2026, S. 2). Geprüft wird H4 primär über den Friedman-Test des qualitativen Gesamtscores über die drei Prompt-Level Low, Medium und High. Paarweise Post-hoc-Vergleiche werden nur dann durchgeführt, wenn der globale Friedman-Test einen signifikanten Unterschied zwischen den Prompt-Levels zeigt.

2. Technologische Fundierung und Begründung des Studiendesigns

2.1 Wahl der Programmiersprache und der Analyse-Infrastruktur

Die Entscheidung, das gesamte Experimentier-Framework sowie die zu generierenden Zielaufgaben in der Programmiersprache Python zu realisieren, ist methodisch durch deren dominierende Rolle in der KI-Entwicklung begründet. Da etablierte Industrie-Benchmarks primär auf Python basieren, ist gewährleistet, dass die zu evaluierenden Sprachmodelle über ein maximal ausgeprägtes technisches Verständnis für diese Syntax verfügen. Dies minimiert das Risiko, dass die Messergebnisse durch mangelnde Trainingsdaten der Modelle verfälscht werden.

Darüber hinaus bietet das Python-Ökosystem eine passgenaue Infrastruktur für die automatisierte statische Code-Analyse. Über das native Modul des abstrakten Syntaxbaums lassen sich spezifische Heuristiken zur Erkennung von Code-Halluzinationen implementieren. Die Eignung dieses Ansatzes ergibt sich daraus, dass „The ast module helps Python applications to process trees of the Python abstract syntax grammar“ (Python Software Foundation 2026, o. S.). Ergänzend kann der abstrakte Syntaxbaum als strukturelle Repräsentation von Quellcode verstanden werden, da „The abstract syntax tree (AST), a fundamental code feature, illustrates the syntactic information of the source code“ (Sun et al. 2023, S. 1).

Für die Erhebung der Software-Qualitätsmetriken wird die Bibliothek Radon eingesetzt. Radon eignet sich hierfür, da „Radon is a Python tool that computes various metrics from the source code“ (Radon 2024, o. S.). Konkret unterstützt das Tool unter anderem „McCabe’s complexity, i.e. cyclomatic complexity“ sowie den „Maintainability Index“ (Radon 2024, o. S.). Die dynamische Testausführung wird über das Test-Framework pytest orchestriert, wobei die JSON-Report-Integration eine strukturierte, maschinenlesbare Weiterverarbeitung der Rohdaten ermöglicht. Dies wird durch das Plugin pytest-json-report gestützt, das beschreibt: „This pytest plugin creates test reports as JSON“ und „This makes it easy to process test results in other applications“ (pytest-json-report 2024, o. S.).

Diese Verzahnung stellt sicher, dass statische Codeanalyse, Qualitätsmetrik-Erhebung und dynamische Testausführung in einer einheitlichen Evaluationspipeline zusammengeführt werden können. Fehlgeschlagene Analysen führen dabei durch defensive Fehlerbehandlung nicht zum Abbruch der Pipeline, sondern werden als nicht-berechenbare Werte protokolliert.

2.2 Wahl der Large Language Models und provider-spezifischer Architekturen

Das Studiendesign ist als symmetrisches dreidimensionales Raster angelegt, dessen erste unabhängige Variable durch die Wahl der Sprachmodelle definiert wird. Um eine breite Abdeckung führender proprietärer Frontier-Systeme zu erreichen, wurden Modelle dreier

zentraler Anbieter ausgewählt: OpenAI Codex in der Spezifikation gpt-4.1 beziehungsweise o3, Anthropic Claude Sonnet 4.6 (mit Claude Haiku 4.5 als Hilfsmodell) sowie Google Gemini CLI mit gemini-3-flash-preview als Hauptmodell und gemini-3.1-flash-lite als Hilfsmodell. Die Auswahl von OpenAI Codex wird dadurch gestützt, dass Codex als „OpenAI's coding agent for software development“ beschrieben wird (OpenAI 2026a, o. S.). Die Einbeziehung von o3 ist zudem aufgrund seiner Positionierung als Reasoning-Modell für anspruchsvolle Programmieraufgaben plausibel, da OpenAI o3 als Modell beschreibt, das „the frontier across coding, math, science, visual perception, and more“ verschiebt (OpenAI 2025, o. S.). Für Anthropic wird Claude als „family of state-of-the-art large language models“ eingeordnet (Anthropic 2026a, o. S.).

Diese Triade dient nicht nur der Abdeckung unterschiedlicher Anbieter, sondern reduziert zugleich das Risiko eines herstellereigenen Bias. Die Einbeziehung genau dieser Modelle ist zudem für die Untersuchung der Token-Effizienz relevant, da sich die zugrundeliegenden Verbrauchs- und Caching-Strukturen deutlich unterscheiden. Während Anthropic eine explizite Trennung zwischen geschriebenen und gelesenen Cache-Tokens vornimmt, indem `cache_creation_input_tokens` als „Number of tokens written to the cache when creating a new entry“ und `cache_read_input_tokens` als „Number of tokens retrieved from the cache for this request“ definiert werden (Anthropic 2026b, o. S.), weist OpenAI Cache-Treffer über das Feld `cached_tokens` innerhalb der Nutzungsmetadaten aus (OpenAI 2026b, o. S.).

Auch Google Gemini stellt nutzungsbezogene Metadaten bereit, da die Antwortstruktur ein Feld `usageMetadata` enthält (Google 2026b, o. S.). Zusätzlich ist die Token-Erfassung kostenrelevant, da „the cost of a call to the Gemini API is determined in part by the number of input and output tokens“ (Google 2026a, o. S.). Für Gemini 2.5 und neuere Modelle wird außerdem implizites Caching beschrieben; Cache-Treffer können im Feld `usage_metadata` der Antwort eingesehen werden (Google 2026c, o. S.). Diese architektonische Diversität macht eine methodische Normalisierung der Token- und Kostenmetriken erforderlich und ermöglicht erst dadurch einen fairen Vergleich der ökonomischen Tragfähigkeit agenteller Reparaturschleifen.

Methodisch ist anzumerken, dass die Evaluation auf der Ebene der agentischen CLIs erfolgt und nicht auf der Ebene isolierter Basismodelle. Sowohl Claude Code als auch Gemini CLI orchestrieren intern jeweils ein Haupt- und ein Hilfsmodell; die Befunde dieser Untersuchung gelten daher für die Agentensysteme in der untersuchten Konfiguration, nicht ausschließlich für die zugrundeliegenden Basismodelle.

2.3 Operationalisierung von Agentenmodus und Prompt-Qualität

Die zweite und dritte unabhängige Variable des Studiendesigns bilden der Agentenmodus sowie die Prompt-Qualität. Der Agentenmodus wird binär in eine direkte Generierung und einen Self-Refining-Prozess unterteilt. Letzterer simuliert einen autonomen Entwicklungszyklus, in dem der Agent seinen eigenen Entwurf bewertet und modifiziert. Um den Einfluss der menschlichen Steuerung zu quantifizieren, wurde ein stufenweiser Prompt-Katalog entwickelt. Die Stufe Low simuliert einen Citizen Developer mit vagen,

umgangssprachlichen Anweisungen ohne Systemkontext. Die Stufe Medium repräsentiert einen Standard-Entwickler, der präzise Signaturen und einzelne Randfälle definiert. Die Stufe High entspricht dem Vorgehen eines Prompt Engineers, der Systemrollen vergibt, Denkstrategien erzwingt und strikte Constraints anlegt. Durch die systematische Kombination von vier Aufgaben, vier Aufgabenvarianten, drei Prompt-Stufen und zwei Architekturmodi entsteht je Agent ein gepaartes Raster aus sechsundneunzig Konfigurationen. Über die drei Agenten ergibt sich daraus ein Gesamtdatensatz von zweihundertachtundachtzig Läufen, der die statistische Validierung der formulierten Hypothesen über nichtparametrische Testverfahren zulässt.

3. Methodisches Vorgehen im Detail: Qualitätsbewertungsmethoden

Um die generierten Code-Artefakte der KI-Agenten jenseits der reinen Funktionsfähigkeit zu evaluieren, erfordert die Untersuchung ein Set an präzisen statischen und heuristischen Metriken. Die folgenden Analyseverfahren bilden den Kern der erweiterten Evaluationspipeline und dienen der Quantifizierung von Effizienz, Wartbarkeit und Modellspezifika. Die Eignung von Radon für diese Aufgabe ergibt sich daraus, dass „Radon is a Python tool which computes various code metrics“ und unter anderem „raw metrics: SLOC, comment lines, blank lines, &c.“ sowie „Cyclomatic Complexity (i.e. McCabe’s Complexity)“ unterstützt (Radon 2020, o. S.).

3.1 Source Lines of Code (SLOC) und Lines of Code (LOC)

Die grundlegendste Metrik zur Erfassung der Code-Länge bildet die Unterscheidung zwischen regulären Lines of Code (LOC) und Source Lines of Code (SLOC). In der Radon-Dokumentation wird LOC als „The total number of lines of code“ definiert, während SLOC als „The number of source lines of code“ beschrieben wird (Radon 2024, o. S.). Zusätzlich werden Kommentar- und Leerzeilen separat ausgewiesen, etwa durch „Blanks: The number of blank lines (or whitespace-only ones)“ (Radon 2024, o. S.). Der Zusammenhang der Rohmetriken wird durch die Gleichung „SLOC + Multi + Single comments + Blank = LOC“ beschrieben (Radon 2024, o. S.).

Im Kontext der Evaluation von Large Language Models dient der bereinigte SLOC-Wert als Maß für die Verbosity eines Modells. Wenn beispielsweise zwei unterschiedliche Agenten-Konfigurationen dieselbe algorithmische Aufgabe fehlerfrei lösen, der eine Agent dafür jedoch zehn Codezeilen und der andere fünfundvierzig Zeilen benötigt, liefert die SLOC-Metrik einen empirischen Hinweis auf die Kompaktheit des generierten Codes. Diese Interpretation wird in der vorliegenden Untersuchung jedoch nicht isoliert betrachtet, sondern mit funktionaler Korrektheit, Lesbarkeit und Komplexität kombiniert, da kürzerer Code nicht zwangsläufig qualitativ besser sein muss.

3.2 Zyklomatische Komplexität nach McCabe

Zur Bewertung der algorithmischen Verschachtelung wird die zyklomatische Komplexität nach McCabe erhoben. Radon definiert diese Metrik wie folgt: „Cyclomatic Complexity corresponds to the number of decisions a block of code contains plus 1“ (Radon 2024, o. S.). Zudem wird der Wert als McCabe-Zahl eingeordnet, die „equal to the number of linearly independent paths through the code“ ist (Radon 2024, o. S.). Die Berechnung erfolgt somit über Kontrollflussstrukturen: Ein lineares Skript ohne zusätzliche Verzweigungen besitzt eine Basis-Komplexität von eins, während jede relevante Entscheidungsstruktur den Wert erhöht.

In der Softwaretechnik ist die zyklomatische Komplexität insbesondere für Test- und Wartbarkeitsfragen relevant. Radon weist darauf hin, dass die Metrik „as a guide when testing conditional logic in blocks“ genutzt werden kann (Radon 2024, o. S.). Ergänzend beschreibt IBM, dass Programme mit niedrigerer zyklomatischer Komplexität „easier to understand and less risky to modify“ sind (IBM 2026, o. S.). Bezogen auf das vorliegende

LLM-Experiment bedeutet ein hoher Komplexitätswert daher, dass ein Agent eine stärker verschachtelte oder kontrollflussintensive Lösung generiert hat. Ein niedrigerer Wert spricht hingegen für einen geradlinigeren Lösungsansatz, sofern die funktionale Korrektheit und semantische Lesbarkeit erhalten bleiben.

3.3 Maintainability Index (Wartbarkeitsindex)

Um einen ganzheitlichen Blick auf die Code-Qualität zu werfen, greift die Pipeline auf den Maintainability Index (MI) zurück. Hierbei handelt es sich um eine zusammengesetzte Metrik, die über eine standardisierte mathematische Formel drei verschiedene Faktoren kombiniert: die zuvor beschriebene zyklomatische Komplexität, die Source Lines of Code (SLOC) sowie das Halstead-Volumen, das nach Halstead (1977, S. 19 ff.) auf Basis der Anzahl unterschiedlicher Operatoren und Operanden sowie ihrer Gesamthäufigkeit definiert ist und das informationstechnische Vokabular des Codes quantifiziert. Das Ergebnis dieser Berechnung wird auf einer Skala von null bis einhundert ausgedrückt, wobei Werte über zwanzig in der Regel als gut wartbar klassifiziert werden und Werte darunter auf problematischen Code hinweisen. Der Maintainability Index fungiert als standardisierter Einzelwert für die Gesamtqualität und beantwortet die essenzielle Frage, wie aufwendig es für einen menschlichen Entwickler wäre, den LLM-generierten Code zu verstehen, anzupassen oder zu reparieren. Durch diese Metrik werden Modelle systematisch abgestraft, die zwar funktionsfähigen, aber übermäßig kryptischen oder strukturell aufgeblähten Code produzieren.

3.4 Hallucination Flags und LLM-spezifische Heuristiken

Neben den klassischen Analyseverfahren für Software erfordert die Arbeit mit Sprachmodellen den Einsatz spezifischer Heuristiken, um die syntaktische Sauberkeit und Rollenkonformität des Outputs zu bewerten. Hierfür wird der generierte Code über einen abstrakten Syntaxbaum (Abstract Syntax Tree, AST) geparkt. Die technische Grundlage hierfür bildet das native Python-Modul `ast`, das laut Dokumentation dazu dient, „trees of the Python abstract syntax grammar“ zu verarbeiten (Python Software Foundation 2026, o. S.). Der Parsing-Vorgang erzeugt dabei eine baumartige Repräsentation des Quellcodes, da „The result will be a tree of objects whose classes all inherit from `ast.AST`“ (Python Software Foundation 2026, o. S.).

In dieser Struktur wird automatisiert nach unerwünschten Elementen gesucht. Zunächst werden unnötige Imports identifiziert, also Fälle, in denen ressourcenintensive Bibliotheken wie `numpy` oder `math` geladen wurden, obwohl die Aufgabenstellung explizit mit Standardmitteln lösbar war. Zudem wird geprüft, ob das Modell unerwünschte `print`-Statements im Code belassen hat, was als Hinweis auf eine unvollständige Trennung zwischen erklärendem Antwortverhalten und strikt ausführbarem Code interpretiert werden kann. Die technische Umsetzbarkeit solcher Prüfungen ergibt sich daraus, dass AST-Strukturen systematisch traversiert werden können; die Python-Dokumentation beschreibt hierfür eine „node visitor base class that walks the abstract syntax tree and calls a visitor function for every node found“ (Python Software Foundation 2026, o. S.).

Ein weiterer kritischer Prüfpunkt ist die Sprachkonsistenz. Es wird analysiert, ob das Modell natürliche Sprachschnipsel oder rudimentäre Markdown-Formatierungen direkt in das auszuführende Skript eingefügt hat. Die Relevanz dieser Prüfung ergibt sich aus der Beobachtung, dass LLM-generierter Code nicht zwangsläufig fehlerfrei ist. Agarwal et al. weisen darauf hin, dass „LLMs are prone to generate hallucinations“ und dabei Inhalte erzeugen können, die plausibel wirken, aber inkorrekt sind (Agarwal et al. 2024, S. 1). Für Codegenerierung ist dies besonders relevant, da generierter Code laut Agarwal et al. „syntactical or logical errors“ enthalten kann (Agarwal et al. 2024, S. 2). Auch Tambon et al. identifizieren in LLM-generiertem Code unter anderem die Bugmuster „Syntax Error“, „Prompt-biased code“ und „Hallucinated Object“ (Tambon et al. 2024, S. 1).

Die Relevanz dieser Metriken begründet sich außerdem in der Modellnutzung als dialogorientierte Systeme. OpenAI beschreibt ChatGPT beispielsweise als Modell, das „interacts in a conversational way“ (OpenAI 2022, o. S.). Gleichzeitig zeigen Ouyang et al., dass größere Sprachmodelle nicht automatisch besser darin sind, der Nutzerintention zu folgen, da „Making language models bigger does not inherently make them better at following a user's intent“ (Ouyang et al. 2022, S. 1). Vor diesem Hintergrund dienen die Heuristiken dazu, zu prüfen, welche Agenten sich in automatisierten Pipelines zuverlässig an die Rolle eines reinen Codegenerators halten und welche zu Formatierungsbrüchen, erklärenden Artefakten oder codebezogenen Halluzinationen neigen.

3.5 Aggregierter qualitativer Gesamtscore

Die in den Abschnitten 3.1 bis 3.4 vorgestellten Einzelmetriken erfassen jeweils nur einen Teilaspekt der Codequalität. Um die unterschiedlichen Dimensionen in einer einzigen, vergleichbaren Kennzahl zusammenzuführen, wird ein zusammengesetzter qualitativer Gesamtscore definiert. Er kombiniert eine binäre Schnittstellenprüfung, eine binäre Constraint-Prüfung, ein heuristisches Minimalitätsmaß und ein Robustheitsmaß als ungewichtetes arithmetisches Mittel:

$$\text{qualitative_score} = (\text{interface_compliance} + \text{constraint_compliance} + \text{minimality_score} + \text{robustness_score}) / 4$$

Die Komponenten sind wie folgt definiert:

$\text{interface_compliance} \in \{0, 1\}$: Der Wert ist 1, wenn alle aufgabenseitig erwarteten Funktionen im generierten Code definiert sind und nicht ausschließlich einen `NotImplementedError` werfen. Andernfalls ist der Wert 0. Diese Komponente prüft, ob der Agent die Schnittstelle der Aufgabenstellung überhaupt umgesetzt hat.

$\text{constraint_compliance} \in \{0, 1\}$: Der Wert ist 1, wenn keine der über den abstrakten Syntaxbaum (Abschnitt 3.4) erfassten Regelverletzungen vorliegt. Geprüft werden unnötige Imports ressourcenintensiver Bibliotheken wie `numpy` oder `math`, verbleibende `print`-Statements, eingebettete natürliche Sprache oder Markdown-Formatierungen sowie Veränderungen der Testdatei. Bereits eine einzige dieser Verletzungen führt zum Wert 0.

$\text{minimality_score} \in [0, 1]$: Der Wert wird heuristisch aus den Rohmetriken SLOC (Abschnitt 3.1) und zyklomatischer Komplexität (Abschnitt 3.2) gebildet. Ausgangswert ist 1,0; für

SLOC zwischen 50 und 80 wird 0,2 abgezogen, für SLOC oberhalb von 80 werden 0,4 abgezogen. Analog wird für eine zyklomatische Komplexität zwischen 9 und 12 ein Abzug von 0,2 vorgenommen, oberhalb von 12 ein Abzug von 0,4. Die Untergrenze liegt bei 0,0. Damit werden überlange oder stark verschachtelte Lösungen systematisch abgestraft, ohne den Maintainability Index direkt in den Score zu übernehmen, der bereits ein eigenes Aggregat aus SLOC, Komplexität und Halstead-Volumen darstellt.

$\text{robustness_score} \in [0, 1]$: Der Wert entspricht der Hidden-Test-Pass-Rate, sofern Hidden Tests für die jeweilige Aufgabe vorhanden sind. Liegen keine Hidden Tests vor, wird die sichtbare Pass-Rate als Rückfallwert verwendet. Diese Komponente bringt eine reale Testevidenz in den Score ein und verhindert, dass syntaktisch saubere, aber funktional fragile Lösungen einen zu hohen Wert erreichen.

Die Wahl eines ungewichteten Mittels ist methodisch begründet: Eine empirisch fundierte Gewichtung würde voraussetzen, dass die relative Bedeutung der vier Dimensionen vorab bekannt ist, was im untersuchten Kontext nicht der Fall ist. Eine gleichmäßige Gewichtung ist daher die konservative und transparente Wahl. Der resultierende Wert liegt auf einer Skala von 0 bis 1 und ist sowohl modell- als auch konfigurationsübergreifend vergleichbar.

Inhaltlich beantwortet der qualitative Gesamtscore eine einzige Frage: Wie nah liegt das generierte Codeartefakt im Mittel an einer Lösung, die formal vollständig ist, regelkonform produziert wurde, kompakt bleibt und auch nicht sichtbare Testfälle besteht? Damit dient der Score nicht als alleinige Entscheidungsgrundlage, sondern als verdichtete Kennzahl, die die Diskussion einzelner Dimensionen in den Abschnitten 6.3 und 6.4 ergänzt. Eine Beispielrechnung zeigt dies plastisch: Eine Konfiguration mit vollständiger Schnittstelle (1,000), eingehaltenen Constraints (1,000), Minimalitätsmaß 0,960 und einer Hidden-Pass-Rate von 0,929 erreicht einen qualitativen Gesamtscore von $(1,000 + 1,000 + 0,960 + 0,929) / 4 = 0,972$; dieser Wert entspricht dem in Tabelle 4 für Claude berichteten Befund.

4. Statistische Methodik

Die im Folgenden berichteten Vergleiche stützen sich auf nichtparametrische Testverfahren in einem gepaarten Studiendesign. Diese Wahl ist aus drei Gründen methodisch geboten. Erstens sind zentrale Zielgrößen wie Test-Pass-Rate, Hidden-Pass-Rate oder qualitativer Score auf das Intervall $[0, 1]$ beschränkt und damit nicht annähernd normalverteilt; viele Konfigurationen treffen die obere Grenze und erzeugen rechtsseitig zensierte Verteilungen. Zweitens enthalten die Tokenverteilungen extreme Ausreißer, weil agentische CLIs bei sehr langen Self-Refining-Schleifen vielfache Kontextwiederholungen erzeugen. Drittens bearbeiten alle drei Agenten dieselben sechsundneunzig Konfigurationen, sodass jeder Vergleich blockweise über identische Aufgaben-, Varianten-, Prompt-Level- und Modus-Kombinationen erfolgt. Eine ungepaarte Auswertung würde diese Strukturierung ignorieren und die Teststärke unnötig senken.

4.1 Friedman-Test

Der Friedman-Test ist ein nichtparametrisches Verfahren für den Vergleich von mehr als zwei gepaarten Stichproben. Er wird als nichtparametrische Alternative zur einfaktoriellen Varianzanalyse mit Messwiederholung beziehungsweise als Test für vollständige Blockdesigns eingesetzt (Friedman 1937, S. 675 ff.; SciPy Developers 2026a, o. S.). Die Grundidee besteht darin, die Werte innerhalb jedes Blocks zu rangieren und anschließend zu prüfen, ob sich die Rangsummen der Gruppen systematisch unterscheiden. Die Nullhypothese lautet, dass die wiederholten Messungen beziehungsweise Behandlungen aus derselben Verteilung stammen und keine systematischen Unterschiede zwischen den Gruppen bestehen (Friedman 1937, S. 675 ff.; SciPy Developers 2026a, o. S.).

In der vorliegenden Arbeit dient der Friedman-Test dem globalen Vergleich der drei Agentensysteme über metrische oder ordinale Zielgrößen wie LOC, SLOC, Maintainability Index, Hidden-Pass-Rate, normalisierte Kontexttokens, Cache Ratio oder qualitativen Gesamtscore. Zusätzlich wird er für den globalen Vergleich der drei Prompt-Level Low, Medium und High verwendet. Die Aussagekraft des Friedman-Tests beschränkt sich bewusst auf die Frage, ob überhaupt ein Unterschied zwischen den Gruppen besteht. Welche Gruppen sich konkret unterscheiden, beantwortet er nicht; hierfür sind paarweise Post-hoc-Vergleiche erforderlich. Ein nicht signifikantes Friedman-Ergebnis ist insbesondere bei den funktionalen Metriken aufschlussreich, weil es den Ceiling-Effekt belegt, der die Ausgangsthese dieser Arbeit motiviert.

4.2 Cochran-Q-Test

Für binäre Zielgrößen wie Success-Binary, Interface Compliance, Constraint Compliance oder das Hallucination Flag wird der Cochran-Q-Test eingesetzt. Er ist für k gepaarte binäre Stichproben geeignet und prüft, ob die Erfolgswahrscheinlichkeiten beziehungsweise Trefferquoten über alle Gruppen hinweg identisch sind (Cochran 1950, S. 256 ff.; statsmodels Developers 2026a, o. S.). Die Nullhypothese lautet, dass alle betrachteten Bedingungen dieselbe Erfolgswahrscheinlichkeit besitzen. Die Alternativhypothese besagt, dass mindestens zwei Bedingungen unterschiedliche Erfolgswahrscheinlichkeiten aufweisen (Cochran 1950, S. 256 ff.).

Der Cochran-Q-Test ist für diese Arbeit besonders relevant, weil mehrere Zielgrößen binär codiert sind, etwa „bestanden/nicht bestanden“, „Constraint eingehalten/nicht eingehalten“ oder „Hallucination Flag vorhanden/nicht vorhanden“. Inhaltlich beantwortet Cochran-Q die Frage, ob ein Agent oder ein Prompt-Level bei solchen binären Zielgrößen eine andere Trefferquote erzielt als die übrigen. Methodisch ist Cochran-Q die k-Gruppen-Erweiterung des McNemar-Tests und kann zugleich als binärer Spezialfall des Friedman-Tests verstanden werden (Cochran 1950, S. 256 ff.; statsmodels Developers 2026a, o. S.). Dadurch ist er für die gepaarte Struktur des Experiments geeignet, in dem dieselbe Konfiguration jeweils von mehreren Agenten oder Prompt-Leveln bearbeitet wird.

4.3 Wilcoxon-Signed-Rank-Test

Der Wilcoxon-Signed-Rank-Test ist ein nichtparametrischer Test für zwei gepaarte Stichproben. Er wird eingesetzt, wenn dieselben Untersuchungseinheiten unter zwei Bedingungen gemessen werden und die Differenzen nicht sinnvoll über einen parametrischen t-Test für gepaarte Stichproben ausgewertet werden sollen (Wilcoxon 1945, S. 80 ff.; SciPy Developers 2026b, o. S.). Die Berechnung basiert auf den vorzeichenbehafteten Rängen der paarweisen Differenzen. Geprüft wird, ob die Verteilung der Differenzen symmetrisch um null liegt beziehungsweise ob sich die beiden Bedingungen systematisch unterscheiden (Wilcoxon 1945, S. 80 ff.; SciPy Developers 2026b, o. S.).

In dieser Arbeit kommt der Wilcoxon-Signed-Rank-Test an zwei Stellen zum Einsatz. Erstens wird er für paarweise Post-hoc-Vergleiche zwischen Agenten verwendet, wenn ein globaler Friedman-Test auf einen signifikanten Unterschied hinweist. Zweitens wird er für den direkten Vergleich der Modi Direct und Self-Refining auf identischen Konfigurationen eingesetzt. Die Stärke des Tests liegt darin, dass er die gepaarte Struktur der Daten berücksichtigt und keine Normalverteilung der Differenzen voraussetzt. Seine Grenze besteht darin, dass er primär systematische Lage- beziehungsweise Tendenzunterschiede erfasst; ein nicht signifikantes Ergebnis schließt Unterschiede in Streuung oder Verteilungsform nicht vollständig aus.

4.4 McNemar-Test

Für paarweise Vergleiche binärer Zielgrößen wird der McNemar-Test eingesetzt. Er ist für zwei gepaarte dichotome Messungen geeignet und prüft, ob die marginalen Erfolgswahrscheinlichkeiten beider Bedingungen gleich sind (McNemar 1947, S. 153 ff.; statsmodels Developers 2026b, o. S.). Die Daten werden in einer 2×2-Tafel angeordnet. Entscheidend für den Test sind nicht die konkordanten Paare, in denen beide Bedingungen gleich abschneiden, sondern die diskordanten Paare, in denen genau eine Bedingung erfolgreich ist und die andere nicht (McNemar 1947, S. 153 ff.; Fagerland et al. 2013, o. S.).

In der vorliegenden Arbeit ist der McNemar-Test insbesondere für paarweise Agentenvergleiche bei binären Metriken relevant, etwa bei Success Rate, Constraint Compliance oder Hallucination Flags. Er beantwortet die Frage, ob ein Agent in genau denjenigen Konfigurationen, in denen sich zwei Agenten unterscheiden, systematisch häufiger erfolgreich ist als der andere. Die Stärke des Verfahrens liegt darin, dass es die gepaarte Datenstruktur nutzt und konkordante Paare, die keine diskriminierende Information

enthalten, nicht zur Testentscheidung heranzieht (McNemar 1947, S. 153 ff.; Fagerland et al. 2013, o. S.).

4.5 Holm-Bonferroni-Korrektur

Bei drei Agenten ergeben sich pro Metrik drei paarweise Vergleiche. Dasselbe gilt für drei Prompt-Level. Über mehrere Metriken hinweg steigt dadurch die Anzahl der Hypothesentests deutlich an. Ohne Korrektur steigt die Wahrscheinlichkeit, mindestens einen falsch positiven Befund zu erhalten, über das nominelle Signifikanzniveau hinaus. Diese Wahrscheinlichkeit wird als Familywise Error Rate bezeichnet (Holm 1979, S. 65 ff.; Eichstaedt et al. 2013, S. 693 ff.).

Zur Kontrolle dieser Fehlerwahrscheinlichkeit wird die Holm-Bonferroni-Korrektur verwendet. Dabei werden die p-Werte aufsteigend sortiert und schrittweise mit angepassten Signifikanzschwellen verglichen. Im Vergleich zur klassischen Bonferroni-Korrektur ist das Verfahren weniger konservativ, kontrolliert aber dennoch die Familywise Error Rate (Holm 1979, S. 65 ff.; Eichstaedt et al. 2013, S. 693 ff.). Inhaltlich bedeutet ein nach Holm korrigiert signifikanter Unterschied, dass der Befund auch unter Berücksichtigung der Mehrfachvergleiche robust bleibt und nicht nur durch die Vielzahl getesteter Paarvergleiche erklärbar ist.

4.6 OLS-Regression zur Prüfung von H1

Für die Prüfung von H1 wird ergänzend eine lineare OLS-Regression geschätzt. Die abhängige Variable ist die Test-Pass-Rate. Als zentrale erklärende Variable wird der logarithmierte normalisierte Tokenverbrauch verwendet. Zusätzlich werden Dummy-Variablen für Architekturmodus und Modell aufgenommen, um deren Einfluss statistisch zu kontrollieren. Ordinary Least Squares ist ein lineares Regressionsverfahren, bei dem die abhängige Variable durch eine oder mehrere erklärende Variablen modelliert wird (statsmodels Developers 2026c, o. S.). Die Logarithmierung ist methodisch begründet, weil ein abnehmender Grenznutzen nichtlinear verläuft: Zusätzliche Tokens sollten zunächst einen stärkeren und später einen geringeren marginalen Beitrag liefern. Diese Annahme knüpft an Skalierungsgesetze für Sprachmodelle an, nach denen Leistungsmaße nicht linear, sondern in Form von Potenzgesetzen mit Modellgröße, Datenmenge und Rechenaufwand zusammenhängen (Kaplan et al. 2020, S. 1).

Die Aussagekraft der Regression liegt in dieser Arbeit nicht primär in der punktgenauen Vorhersage einzelner Läufe, sondern in zwei aggregierten Kennzahlen. Erstens zeigt der Koeffizient des logarithmierten Tokenverbrauchs, ob zusätzlicher Tokeneinsatz überhaupt einen messbaren marginalen Beitrag zur Test-Pass-Rate leistet. Zweitens zeigt das Bestimmtheitsmaß R^2 , welcher Anteil der Varianz der Test-Pass-Rate durch das Regressionsmodell erklärt wird. Ein Koeffizient nahe null, ein hoher p-Wert und ein sehr niedriges R^2 sprechen dafür, dass zusätzlicher Tokenverbrauch in dem untersuchten Aufgabenraum keine substanzielle Mehrleistung mehr erzeugt. In der vorliegenden Untersuchung wird dies als Hinweis auf einen Sättigungseffekt interpretiert, der zur Hypothese eines abnehmenden Grenznutzens passt.

4.7 Interpretationsrahmen

Die berichteten p-Werte werden im Folgenden als Entscheidungsgrundlage gegen ein Signifikanzniveau von $\alpha = 0,05$ interpretiert. Ergebnisse mit $p < 0,001$ werden als hochsignifikant ausgewiesen, Ergebnisse mit p zwischen 0,001 und 0,05 als signifikant und Ergebnisse mit $p \geq 0,05$ als nicht signifikant. Bei paarweisen Post-hoc-Vergleichen wird grundsätzlich der Holm-korrigierte p-Wert berichtet. Die Bewertung der Hypothesen H1 bis H4 erfolgt ausschließlich auf Grundlage dieser Kriterien; Tendenzen ohne statistische Absicherung werden im Fließtext als solche markiert und nicht als Bestätigung gewertet.

5. Methodik zur Normalisierung des Tokenverbrauchs

Da die untersuchten Sprachmodelle und deren Laufzeitumgebungen – namentlich Claude Code, OpenAI Codex und Gemini CLI – divergierende Rohfelder zur Erfassung des Tokenverbrauchs ausgeben, ist ein direkter quantitativer Vergleich der unverarbeiteten Werte methodisch unzulässig. Die anbieterspezifischen Messwerte müssen zwingend auf ein gemeinsames Vergleichsschema abgebildet werden. Die Grundlage dieser Normalisierung bildet die strikte Unterscheidung der Token-Arten. Um diese Datenpunkte modellübergreifend zu harmonisieren, wird die in Tabelle 1 definierte Zielstruktur für alle aggregierten Ergebnisse angewendet.

Feld	Beschreibung
<code>prompt_tokens_incl_cache</code>	Der gesamte promptseitige Kontext, der vom Modell verarbeitet wurde, bestehend aus neuen Eingaben und wiederverwendeten Cache-Inhalten.
<code>uncached_input_tokens</code>	Die regulären Eingabe-Tokens, die bei der aktuellen API-Anfrage neu berechnet (bzw. in den Cache geschrieben) werden mussten.
<code>cache_read_tokens</code>	Die Tokens, die erfolgreich und kostengünstiger aus dem bestehenden Kontext-Cache des Modells geladen wurden.
<code>completion_tokens</code>	Die vom Modell generierten, sichtbaren Ausgabe-Tokens, die den eigentlichen Quellcode oder Erklärtext bilden.
<code>reasoning_tokens</code>	Verborgene, interne Denk-Tokens (Thoughts), die insbesondere bei fortgeschrittenen Modellen zur Problemlösung generiert werden.
<code>total_tokens_normalized</code>	Die harmonisierte Summe aller verarbeiteten Tokens; dient als technische Vergleichsgröße für die Effizienzanalyse.

Tabelle 1: Definition der normalisierten Token-Felder für die vergleichende Evaluation

5.1 Allgemeine Normalisierungsformeln und Kennzahlen

Für alle evaluierten Tools wird die normalisierte Gesamtzahl der Tokens mathematisch definiert als die Summe aus dem gesamten Prompt-Kontext, den Ausgaben und den internen Denkprozessen. Diese Metrik fungiert primär als technische Verbrauchskennzahl.

Formel 1: Normalisierte Gesamt-Tokens

$$Total_Tokens_{\text{normalized}} = Prompt_Tokens_incl_Cache + Completion_Tokens + Reasoning_Tokens$$

Zur Bewertung der Wiederverwendung von Kontext – was insbesondere bei agentischen Tools im Self-Refining-Modus relevant ist – wird die Cache-Ratio ermittelt. Diese Kennzahl beschreibt den Anteil der aus dem Cache geladenen Tokens am gesamten Prompt.

Formel 2: Cache-Ratio

$$Cache_Ratio = \frac{Cache_Read_Tokens}{Prompt_Tokens_incl_Cache}$$

Um die Effizienz der Modelle direkt mit der funktionalen Lösungsquote in Relation zu setzen, wird der normalisierte Gesamtverbrauch durch die Anzahl der erfolgreich bestandenen Testfälle dividiert.

Formel 3: Tokens pro Erfolg

$$Tokens_per_Success = \frac{Total_Tokens_{\text{normalized}}}{Anzahl_bestandener_Tests}$$

5.2 Normalisierung der Token-Metriken für Anthropic Claude

Im Falle von Claude Code ist eine additive Rekonstruktion des gesamten Promptumfangs notwendig, da Anthropic den Inputverbrauch strikt in getrennte Kategorien unterteilt. Der gesamte Prompt setzt sich aus regulären Eingaben, neu erstellten Cache-Tokens und gelesenen Cache-Tokens zusammen. Da Claude in der genutzten Version keine separaten Reasoning-Tokens ausweist, wird dieser Wert auf null gesetzt.

Berechnungsgrundlage für Claude Code:

$$Prompt_Tokens_incl_Cache = Input_Tokens + Cache_Creation_Tokens + Cache_Read_Tokens$$

$$Total_Tokens_{\text{normalized}} = Prompt_Tokens_incl_Cache + Output_Tokens + 0$$

Anhand der Log-Daten eines referenzierten Testlaufs ergibt sich folgende Beispielrechnung: 9 reguläre Input-Tokens addiert mit 9.123 Cache-Creation-Tokens und 148.355 Cache-Read-Tokens führen zu einem Promptumfang von 157.487 Tokens. Zuzüglich der 1.505 Ausgabe-Tokens resultiert ein normalisierter Gesamtverbrauch von 158.992 Tokens.

5.3 Normalisierung der Token-Metriken für OpenAI Codex

Bei der Auswertung der Logs von OpenAI Codex (in der CLI-Default-Konfiguration) ist eine abweichende Methodik erforderlich. Hier sind die gecachten Tokens bereits als Teilmenge in den gesamten Input-Tokens enthalten. Eine Addition würde zu einer verfälschenden Doppelzählung führen. Um den nicht-gecachten Anteil zu ermitteln, muss eine Subtraktion erfolgen. Zudem müssen die explizit ausgewiesenen Reasoning-Tokens addiert werden.

Berechnungsgrundlage für OpenAI Codex:

$$Prompt_Tokens_incl_Cache = Input_Tokens_Total$$

$$Uncached_Input_Tokens = Input_Tokens_Total - Cached_Tokens$$

$$Total_Tokens_{normalized} = Input_Tokens_Total + Output_Tokens + Reasoning_Tokens$$

In einem dokumentierten Lauf mit 48.898 gesamten Input-Tokens (wovon 41.984 gecacht waren), 921 Output-Tokens und 24 Reasoning-Tokens beläuft sich der normalisierte Gesamtverbrauch demnach direkt auf 49.843 Tokens.

5.4 Normalisierung der Token-Metriken für Google Gemini CLI

Die Normalisierung für die Google Gemini CLI erfordert einen Aggregationsschritt, da der Agent in dem Versuchsaufbau iterativ mehrere Modelle (wie Flash-Lite und Flash-Preview) verwendet. Die Metadaten müssen über alle Modelle hinweg aufsummiert werden (Σ), um den tatsächlichen Agenten-Lauf abzubilden. Google weist hierbei spezifische Felder für Kandidaten (Candidates) und Denkprozesse (Thoughts) aus.

Berechnungsgrundlage für Google Gemini:

$$Prompt_Tokens_incl_Cache = \sum Prompt_Tokens$$

$$Uncached_Input_Tokens = \sum Prompt_Tokens - \sum Cached_Tokens$$

$$Total_Tokens_{normalized} = \sum Prompt_Tokens + \sum Candidate_Tokens + \sum Thought_Tokens$$

Im vorliegenden Gemini-Log wurden gemini-3.1-flash-lite (967 Prompt-Tokens) und gemini-3-flash-preview (47.763 Prompt-Tokens, davon 26.744 gecacht) verwendet. Die Normalisierung erfolgt somit über die Aggregation:

- $Prompt = 967 + 47.763 = 48.730$
- $Candidates = 48 + 1.302 = 1.350$
- $Thoughts = 94 + 2.419 = 2.513$
- $Total_Tokens_{normalized} = 48.730 + 1.350 + 2.513 = 52.593$

Diese exakte mathematische Operationalisierung gewährleistet, dass die spätere statistische Auswertung der Token-Kosten auf einem belastbaren, modellübergreifend harmonisierten Fundament steht.

6. Auswertung der Ergebnisse

Die finale Untersuchung umfasst 288 Experimentläufe. Jeder der drei untersuchten Coding-Agenten Claude, Codex und Gemini bearbeitete 96 identische Konfigurationen. Diese ergeben sich aus vier Aufgaben, vier Varianten, drei Prompt-Leveln und zwei Architekturmodi. Da jede Konfiguration von allen drei Agenten bearbeitet wurde, liegt ein gepaartes Studiendesign vor. Die statistische Auswertung erfolgt daher blockweise über dieselben Aufgaben-Konfigurationen.

Für die Interpretation wird zwischen zwei Analyseebenen unterschieden. Die erste Ebene beschreibt die operative Stabilität des Agentensystems, also ob ein Agent den vollständigen Aufgabenraum technisch stabil bearbeiten kann. Die zweite Ebene beschreibt die Artefaktqualität, also die Qualität tatsächlich erzeugter und auswertbarer Codeartefakte. Diese Trennung ist notwendig, weil Claude nur in 70 von 96 Konfigurationen regulär ausführbar war. In 26 Konfigurationen liegt kein regulärer Output vor. Diese Läufe sind in den Daten durch `agent_exit_code = 1`, `failure_mode = not_implemented`, 0 bestandene Tests und 0 normalisierte Kontexttokens gekennzeichnet.

Die 26 nicht regulären Claude-Läufe bleiben für die operative Vollausswertung relevant, weil ein Agentensystem im praktischen Einsatz auch daran gemessen wird, ob es über den vollständigen Aufgabenraum hinweg stabil ausführbar bleibt. Sie dürfen jedoch nicht als reguläre Codeartefakte in die Bewertung von Wartbarkeit, Minimalität, Hallucination-Verhalten, qualitativer Bewertung oder Token-Effizienz einfließen. Deshalb werden im Agentenvergleich (Abschnitte 6.1 bis 6.4) operative Kennzahlen sowohl für $n = 70$ als auch für $n = 96$ berichtet, während die Artefaktqualität ausschließlich auf der fairen $n = 70$ -Teilstichprobe basiert. Für die Auswertungen des Agentenmodus und der Prompt-Qualität (Abschnitte 6.5 und 6.6) ergeben sich aus derselben fairen Vergleichsbasis abweichende Blockgrößen, die im jeweiligen Abschnitt explizit angegeben werden.

6.1 Operative Auswertung im fairen Bereich $n = 70$

Die erste operative Betrachtung umfasst jene 70 Konfigurationen, in denen Claude regulär ausführbar war. In dieser Teilmenge liefern alle drei Agenten überwiegend funktionsfähige Lösungen. Claude und Gemini erreichen jeweils eine Success Rate von 100,0 %, Codex liegt mit 98,6 % nur minimal darunter. Auch die Test-Pass-Rate ist praktisch gesättigt.

Agent	n	reguläre Läufe	technische Abbrüche	Success Rate	Test Pass Rate	Hidden Pass Rate	Ø Laufzeit	Ø Kontexttokens	Ø Tokens pro bestandenem Testfall
Claude	70	70	0	100,0 %	100,0 %	92,9 %	30,23 s	112.121	6.998

Code x	70	70	0	98,6 %	99,7 %	79,3 %	35,52 s	62.801	3.870
Gemini	70	70	0	100,0 %	100,0 %	83,3 %	51,23 s	256.319	14.978

Für die Hidden-Pass-Rate beträgt die Friedman-Teststatistik $\chi^2 = 8,61$ bei $p = 0,0135$.

Der globale Vergleich der funktionalen Erfolgsrate ist in dieser Teilmenge nicht signifikant. Der Cochran-Q-Test ergibt $Q = 2,00$ bei $p = 0,368$. Auch die Test-Pass-Rate unterscheidet sich nicht signifikant zwischen den drei Agenten; der Friedman-Test ergibt ebenfalls $\chi^2 = 2,00$ bei $p = 0,368$. Damit zeigt sich bereits in der fairen Teilmenge ein deutlicher Ceiling-Effekt: Die reine Pass/Fail-Bewertung kann Claude, Codex und Gemini funktional nicht mehr trennscharf unterscheiden.

Gleichzeitig zeigen operative Sekundärgrößen deutliche Unterschiede. Die Laufzeit unterscheidet sich signifikant zwischen den Agenten, Friedman $\chi^2 = 48,37$, $p = 3,14e-11$. Auch der normalisierte Kontexttokenverbrauch unterscheidet sich hochsignifikant, Friedman $\chi^2 = 134,26$, $p = 7,02e-30$. Codex ist in dieser Perspektive der effizienteste Agent, während Gemini den höchsten Ressourcenverbrauch aufweist.

Die Spalte „Tokens pro bestandem Testfall“ ist als Effizienzkennzahl zu verstehen. Sie berechnet sich pro Lauf als normalisierte Tokenzahl dividiert durch die Anzahl bestandener Testfälle. Sie ist daher nicht identisch mit den durchschnittlichen Kontexttokens pro Konfiguration.

6.2 Operative Vollausswertung n = 96

Die zweite operative Betrachtung umfasst den vollständigen Aufgabenraum mit 96 Konfigurationen je Agent. In dieser Perspektive bleiben auch technische Abbrüche enthalten, weil sie für den praktischen Einsatz eines Agentensystems relevant sind. Hier schneiden Codex und Gemini hinsichtlich operativer Stabilität deutlich besser ab als Claude.

Agent	n	reguläre Läufe	technische Abbrüche	Success Rate	Test Pass Rate	Hidden Pass Rate	Ø Laufzeit beobachtet	Ø Kontexttokens beobachtet
Claude	96	70	26	72,9 %	72,9 %	67,7 %	23,36 s	81.755
Code x	96	96	0	99,0 %	99,8 %	83,1 %	35,44 s	62.677
Gemini	96	96	0	100,0 %	100,0 %	86,3 %	49,37 s	244.994

Der Modellvergleich über alle 96 Konfigurationen ist hochsignifikant. Für die Success Rate ergibt sich ein Cochran-Q-Wert von $Q = 48,22$ bei $p = 3,38e-11$. Die paarweisen McNemar-Tests zeigen, dass Claude in der Vollausswertung signifikant schlechter abschneidet als Codex und Gemini. Die Holm-korrigierten p-Werte liegen bei $p = 8,34e-07$ für Claude vs. Codex und $p = 8,94e-08$ für Claude vs. Gemini. Zwischen Codex und Gemini besteht kein signifikanter Unterschied.

Diese Vollausswertung beschreibt jedoch nicht die Artefaktqualität, sondern die operative Stabilität des gesamten Agentensystems. Die 26 nicht regulären Claude-Läufe zeigen, dass Claude den vollständigen 96er-Aufgabenraum unter den gegebenen Bedingungen nicht stabil abschließt. Die beobachteten Laufzeit- und Tokenwerte der Vollausswertung sind für Claude nur eingeschränkt als Effizienzkennzahlen interpretierbar, da abgebrochene Läufe mit kurzen Laufzeiten und 0 normalisierten Kontexttokens in die Aggregation eingehen. Aus diesem Grund wird in der $n = 96$ -Tabelle kein qualitativer Gesamtscore berichtet. Ein solcher Wert würde technische Abbrüche mit regulärer Artefaktqualität vermischen.

6.3 Artefaktqualität im fairen Vergleich $n = 70$

Die Artefaktqualität wird ausschließlich auf Basis der 70 regulär vergleichbaren Konfigurationen bewertet. Nur in dieser Teilmenge liegen für alle drei Agenten echte Codeartefakte vor. Für metrische bzw. ordinale Metriken wurde der Friedman-Test verwendet. Für binäre Metriken wurde der Cochran-Q-Test verwendet.

Die Variablen `constraint_compliance` und `hallucination_flag` werden gemeinsam betrachtet, da sie in den Daten exakt komplementär codiert sind. Ein separater Test beider Variablen würde dieselbe Evidenz doppelt berichten. Die Variable `language_consistency` wird nicht als eigenständige Qualitätsmetrik interpretiert, da sie in den vorliegenden Daten exakt der binarisierten Hidden-Pass-Information entspricht. Damit misst sie in dieser Auswertung keine unabhängige Sprachkonsistenz, sondern dupliziert eine bereits erfasste Testinformation. Der in der folgenden Tabelle berichtete qualitative Gesamtscore aggregiert die zuvor in Abschnitt 3.5 definierten vier Teildimensionen Interface Compliance, Constraint Compliance, Minimality Score und Robustness Score zu einer einzigen Vergleichsgröße.

Metrik	Claude	Codex	Gemini	Test	p-Wert	Interpretation
Success Rate	1,000	0,986	1,000	Cochran-Q	0,368	nicht signifikant
Test Pass Rate	1,000	0,997	1,000	Friedman	0,368	nicht signifikant
Hidden Pass Rate / Test-Robustheit	0,929	0,793	0,833	Friedman	0,0135	signifikant
LOC	35,46	41,59	48,51	Friedman	$1,01e-23$	hochsignifikant

SLOC	30,50	32,70	37,84	Friedman	2,23e-19	hochsignifika nt
Zyklomatische Komplexität	6,48	5,72	6,83	Friedman	3,39e-08	hochsignifika nt
Maintainability Index	48,89	48,41	52,30	Friedman	0,741	nicht signifikant
Minimality Score	0,960	0,977	0,934	Friedman	0,000590	signifikant
Interface Compliance	1,000	1,000	1,000	nicht testbar	—	keine Varianz
Constraint-/Hallucination-Compliance	1,000	1,000	0,757	Cochran-Q	4,14e-08	hochsignifika nt
Parse Error	0,000	0,000	0,000	nicht testbar	—	keine Varianz
Laufzeit gesamt	30,23 s	35,52 s	51,23 s	Friedman	3,14e-11	hochsignifika nt
Normalisierte Kontexttokens	112.121	62.801	256.319	Friedman	7,02e-30	hochsignifika nt
Tokens pro bestandem Testfall	6.998	3.870	14.978	Friedman	7,02e-30	hochsignifika nt
Cache Ratio	0,910	0,830	0,723	Friedman	1,88e-18	hochsignifika nt
Qualitativer Gesamtscore	0,972	0,942	0,881	Friedman	5,27e-05	hochsignifika nt

Die Artefaktmetriken zeigen damit ein deutlich anderes Bild als die reine Success Rate. Funktional sind die drei Agenten in der n = 70-Teilstichprobe nicht signifikant unterscheidbar. Die sekundären Qualitätsmetriken trennen die Agenten dagegen in mehreren relevanten Dimensionen.

Claude erreicht in der regulären Teilmenge die höchste Hidden-Pass-Rate und den höchsten qualitativen Gesamtscore. Codex erzielt die beste Token-Effizienz, den niedrigsten Tokenverbrauch pro bestandem Testfall und die niedrigste zyklomatische Komplexität. Gemini ist funktional perfekt, erzeugt jedoch längere Artefakte, benötigt deutlich mehr Tokens und weist häufiger Constraint- bzw. Hallucination-Probleme auf. Der Maintainability Index unterscheidet sich dagegen nicht signifikant zwischen den Agenten. Der höhere Mittelwert von Gemini beim Maintainability Index wird daher nicht als belastbarer Qualitätsvorteil interpretiert.

6.4 Post-hoc-Analyse der Artefaktqualität

Nach den globalen Tests wurden paarweise Post-hoc-Vergleiche durchgeführt. Für metrische Metriken wurde der Wilcoxon-Signed-Rank-Test verwendet, für binäre Metriken der McNemar-Test. Die p-Werte wurden nach Holm korrigiert.

Metrik	signifikanter Post-hoc-Befund	Holm-korrigierter p-Wert	Inhaltliche Aussage
Hidden Pass / Test-Robustheit	Claude > Codex	0,00023	Claude besteht Hidden Tests häufiger als Codex
Hidden Pass / Test-Robustheit	Claude > Gemini	0,00836	Claude ist test-robuster als Gemini
Hidden Pass / Test-Robustheit	Gemini > Codex	0,00712	Gemini ist test-robuster als Codex
LOC	Claude < Codex	2,30e-10	Claude erzeugt kürzere Artefakte
LOC	Claude < Gemini	3,41e-12	Claude erzeugt deutlich kürzeren Code als Gemini
LOC	Codex < Gemini	1,71e-09	Codex ist kompakter als Gemini
SLOC	Claude < Codex	2,41e-05	Claude erzeugt weniger ausführbare Codezeilen als Codex
SLOC	Claude < Gemini	1,06e-10	Claude erzeugt deutlich weniger SLOC als Gemini
SLOC	Codex < Gemini	1,06e-10	Codex ist kompakter als Gemini
Zyklomatische Komplexität	Codex < Claude	0,00059	Codex erzeugt einfachere Kontrollflüsse als Claude
Zyklomatische Komplexität	Codex < Gemini	1,61e-06	Codex erzeugt einfachere Kontrollflüsse als Gemini

Minimality Score	Claude > Gemini	0,0251	Claude ist minimaler als Gemini
Minimality Score	Codex > Gemini	0,000824	Codex ist minimaler als Gemini
Constraint-/Hallucination-Compliance	Claude > Gemini	4,58e-05	Gemini verletzt häufiger Vorgaben bzw. zeigt häufiger Hallucination Flags
Constraint-/Hallucination-Compliance	Codex > Gemini	4,58e-05	Gemini verletzt häufiger Vorgaben bzw. zeigt häufiger Hallucination Flags
Laufzeit	Claude < Codex	0,00024	Claude ist schneller als Codex
Laufzeit	Claude < Gemini	1,00e-09	Claude ist deutlich schneller als Gemini
Laufzeit	Codex < Gemini	1,67e-08	Codex ist schneller als Gemini
Kontexttokens	Codex < Claude	1,07e-12	Codex ist token-effizienter als Claude
Kontexttokens	Claude < Gemini	1,07e-12	Claude benötigt weniger Tokens als Gemini
Kontexttokens	Codex < Gemini	1,07e-12	Codex ist deutlich effizienter als Gemini
Tokens pro bestandemem Testfall	Codex < Claude	1,07e-12	Codex benötigt weniger Tokens pro bestandemem Testfall
Tokens pro bestandemem Testfall	Claude < Gemini	1,07e-12	Claude benötigt weniger Tokens pro bestandemem Testfall als Gemini
Tokens pro bestandemem Testfall	Codex < Gemini	1,07e-12	Codex ist deutlich effizienter als Gemini
Cache Ratio	Claude > Codex	1,48e-06	Claude nutzt anteilig mehr Cache als Codex

Cache Ratio	Claude > Gemini	1,07e-12	Claude nutzt anteilig mehr Cache als Gemini
Cache Ratio	Codex > Gemini	3,31e-08	Codex nutzt anteilig mehr Cache als Gemini
Qualitativer Score	Claude > Codex	0,00050	Claude erzielt den höchsten qualitativen Score
Qualitativer Score	Claude > Gemini	0,00018	Claude liegt über Gemini
Qualitativer Score	Codex > Gemini	0,00018	Codex liegt über Gemini

Nicht signifikant sind insbesondere die paarweisen Unterschiede zwischen Claude und Codex beim Minimality Score sowie alle paarweisen Unterschiede beim Maintainability Index. Der Minimality-Effekt wird damit vor allem durch Gemini getrieben. Codex und Claude unterscheiden sich hier nicht belastbar voneinander, beide liegen jedoch signifikant über Gemini.

Bei der zyklomatischen Komplexität ist außerdem der paarweise Unterschied zwischen Claude und Gemini nicht signifikant. Der globale Effekt entsteht hier vor allem dadurch, dass Codex gegenüber beiden anderen Agenten niedrigere Komplexitätswerte erzeugt.

Die Post-hoc-Analyse zeigt vier zentrale Befundgruppen. Erstens ist Claude in den regulär vergleichbaren Läufen hinsichtlich Hidden-Pass-Rate bzw. Test-Robustheit am stärksten. Das widerspricht einer einfachen Interpretation der 96er-Vollauswertung, wonach Claude pauschal das schwächste Modell wäre. Der niedrigere Claude-Wert in der Vollauswertung ist daher primär als operative Stabilitätsproblematik zu verstehen.

Zweitens zeigen die Wartbarkeitsmetriken LOC und SLOC, dass Gemini tendenziell längere Codeartefakte erzeugt. Claude produziert die kompaktesten Lösungen, während Codex bei der zyklomatischen Komplexität am besten abschneidet. Der Maintainability Index unterscheidet sich dagegen nicht signifikant. Die Modelle erzeugen also unterschiedlich umfangreichen und unterschiedlich verschachtelten Code, ohne dass daraus ein statistisch belastbarer Unterschied im aggregierten Wartbarkeitsindex folgt.

Drittens zeigt die kombinierte Constraint-/Hallucination-Betrachtung einen klaren Nachteil von Gemini. Claude und Codex erreichen vollständige Constraint Compliance und zeigen keine Hallucination Flags, während Gemini in 24,3 % der regulären Läufe Hallucination Flags aufweist bzw. nur eine Constraint Compliance von 75,7 % erreicht. Dieser Befund wird nur einmal berichtet, weil beide Variablen in den Daten exakt komplementär codiert sind.

Viertens zeigt die Tokenanalyse den stärksten Effizienzunterschied. Codex benötigt mit durchschnittlich 62.801 normalisierten Kontexttokens und 3.870 Tokens pro bestandem Testfall den geringsten Ressourcenaufwand. Claude liegt mit 112.121 Kontexttokens und 6.998 Tokens pro bestandem Testfall im Mittelfeld. Gemini benötigt mit 256.319 Kontexttokens und 14.978 Tokens pro bestandem Testfall mit Abstand die meisten

Ressourcen. Diese Unterschiede sind hochsignifikant und zeigen, dass funktional gleichwertige Ergebnisse mit sehr unterschiedlichem Aufwand erzeugt werden.

6.5 Auswertung des Agentenmodus: Direct vs. Self-Refining

Neben der Modellwahl wurde der Einfluss des Agentenmodus untersucht. Verglichen wurden der direkte Generierungsmodus und der Self-Refining-Modus. Die Hypothese H2 nahm an, dass Self-Refining durch zusätzliche Selbstkorrektur- und Reparaturschleifen insbesondere die Robustheit gegenüber Hidden Tests erhöht.

Für die Bewertung des Agentenmodus ist erneut zwischen Vollausswertung und fairer Artefaktbetrachtung zu unterscheiden. Die Vollausswertung umfasst 144 Direct-Läufe und 144 Self-Refining-Läufe. Sie beschreibt die operative Wirkung des Modus über den vollständigen Aufgabenraum. Die faire Artefaktbetrachtung umfasst dagegen nur die regulär vergleichbaren Claude-Konfigurationen und damit 105 Läufe je Modus, die sich blockweise zu 105 gepaarten Direct/Self-Refining-Vergleichen zusammenfügen.

Auswertung	Modus	n	Success Rate	Test Pass Rate	Hidden Pass Rate	Ø Laufzeit	Ø Kontexttokens	Ø Tokens pro bestandem Testfall	Ø qualitativer Score
Vollausswertung	Direct	144	90,3 %	90,8 %	79,6 %	35,67 s	128.130	8.731	0,928
Vollausswertung	Self-Refining	144	91,0 %	91,0 %	78,5 %	36,45 s	131.487	8.943	0,922
Faire Artefaktbasis	Direct	105	99,0 %	99,8 %	85,6 %	39,30 s	143.722	8.652	0,935
Faire Artefaktbasis	Self-Refining	105	100,0 %	100,0 %	84,8 %	38,69 s	143.773	8.579	0,929

In der Vollausswertung zeigt Self-Refining nur einen minimalen Vorteil bei Success Rate und Test Pass Rate. Der Unterschied ist statistisch nicht signifikant. Für die Test-Pass-Rate ergibt der Wilcoxon-Test $p = 0,317$; für die binäre Success Rate ergibt der McNemar-Test ebenfalls keinen signifikanten Unterschied. Auch die Hidden Pass Rate verbessert sich nicht signifikant durch Self-Refining, $p = 0,143$. Damit wird der zentrale Wirkmechanismus der Hypothese H2 nicht bestätigt.

Auffällig ist lediglich die Laufzeit in der Vollauswertung. Self-Refining ist mit durchschnittlich 36,45 s etwas langsamer als Direct mit 35,67 s; dieser Unterschied ist statistisch signifikant, $p = 0,0038$. Inhaltlich ist der Effekt jedoch klein. Für den normalisierten Kontexttokenverbrauch ergibt sich kein signifikanter Unterschied, $p = 0,157$. Die Tokenwerte der Vollauswertung sind außerdem wegen der Claude-Abbrüche nur eingeschränkt als Artefakt-Effizienz interpretierbar.

In der fairen Artefaktbasis ($n = 105$ Läufe je Modus) verschwindet selbst der Laufzeitunterschied. Direct und Self-Refining unterscheiden sich dort weder in Success Rate, Test Pass Rate, Hidden Pass Rate, Laufzeit, Tokenverbrauch noch qualitativem Score signifikant. Die Hidden Pass Rate liegt bei Direct bei 85,6 % und bei Self-Refining bei 84,8 %, $p = 0,179$. Der qualitative Score liegt bei Direct bei 0,935 und bei Self-Refining bei 0,929, $p = 0,312$. Der Kontexttokenverbrauch unterscheidet sich ebenfalls nicht signifikant, $p = 0,351$.

Damit wird Hypothese H2 abgelehnt. Self-Refining erzeugt in dieser Untersuchung keinen statistisch belastbaren Robustheitsgewinn. Die zusätzliche Agentenschleife rechtfertigt sich auf Basis der vorliegenden Daten nicht durch bessere funktionale oder qualitative Ergebnisse. Im besten Fall ist Self-Refining neutral, in der Vollauswertung jedoch leicht langsamer.

6.6 Auswertung der Prompt-Qualität: Low, Medium und High

Die dritte Hypothese H3 nahm an, dass eine systematische Erhöhung der Prompt-Qualität den normalisierten Tokenverbrauch je Lauf signifikant senkt, da präzisere Anweisungen den explorativen Anteil der Inferenz reduzieren. Zur Prüfung wurden die Prompt-Level Low, Medium und High über den Friedman-Test der normalisierten Kontexttokens verglichen. Daneben werden in diesem Abschnitt auch die übrigen, nicht zur Hypothese gehörenden Effekte des Prompt-Levels berichtet, um ein vollständiges Bild zu liefern.

In der operativen Vollauswertung über alle 96 Konfigurationen je Prompt-Level ergibt sich zwar ein knapp signifikanter Unterschied bei Success Rate und Test-Pass-Rate, jeweils $p = 0,0498$. Dieser Befund ist jedoch durch die Claude-Abbrüche konfundiert und darf nicht als reiner Prompt-Qualitätseffekt interpretiert werden. Für die eigentliche Bewertung der Prompt-Qualität wird deshalb die faire Artefaktbasis verwendet.

Für die Artefaktbewertung wird erneut die faire Vergleichsmenge verwendet. Da nach Ausschluss der nicht regulären Claude-Konfigurationen nicht für alle Blöcke alle drei Prompt-Level vollständig vorliegen, basiert der gepaarte Friedman-Vergleich auf 66 vollständigen Blöcken je Prompt-Level. Insgesamt gehen damit 198 reguläre Läufe in die Prompt-Analyse ein.

Prompt-Level	n	Success	Test Pass	Hidden Pass	Constraint-/Hallucination-Compliance	SL OC	Ø Laufzeit	Ø Kontexttokens	Ø Tokens pro bestan	Ø qualitativer Score
--------------	---	---------	-----------	-------------	--------------------------------------	-------	------------	-----------------	---------------------	----------------------

		Rat e	Ra te	Rat e					denem Testfall	
Low	6 6	100, 0 %	10 0, 0 %	84,3 %	89,4 %	33, 42	40,9 8 s	154.701	9.468	0,923
Medi um	6 6	100, 0 %	10 0, 0 %	84,2 %	90,9 %	33, 89	39,1 0 s	145.658	8.763	0,928
High	6 6	98,5 %	99 ,7 %	84,6 %	93,9 %	34, 65	37,5 9 s	129.540	7.655	0,933

Die funktionale Leistung unterscheidet sich nicht signifikant zwischen den drei Prompt-Leveln. Für die Success Rate ergibt der Cochran-Q-Test $p = 0,368$; für die Test-Pass-Rate ergibt der Friedman-Test ebenfalls $p = 0,368$. Damit zeigt sich auch bei der Prompt-Qualität ein Ceiling-Effekt: Die sichtbare funktionale Leistung ist nahezu vollständig gesättigt.

Auch die Robustheit gegenüber Hidden Tests verbessert sich nicht signifikant mit höherem Prompt-Level. Die Hidden Pass Rate liegt bei Low bei 84,3 %, bei Medium bei 84,2 % und bei High bei 84,6 %. Der Friedman-Test ergibt $p = 0,695$. Damit lässt sich kein belastbarer Robustheitsgewinn durch bessere Prompts nachweisen.

Für Constraint-/Hallucination-Compliance zeigt sich zwar ein plausibler numerischer Trend: Low erreicht 89,4 %, Medium 90,9 % und High 93,9 %. Der Unterschied ist jedoch statistisch nicht signifikant, $p = 0,558$. Die Annahme, dass High-Level-Prompts Hallucination Flags signifikant reduzieren, wird daher nicht bestätigt. Dieser Punkt ist methodisch wichtig, weil die numerische Verbesserung nicht als belastbarer Effekt überinterpretiert werden darf.

Auch die Codegröße sinkt nicht durch bessere Prompts. Im Gegenteil steigt SLOC numerisch von 33,42 bei Low auf 34,65 bei High. Der Unterschied ist jedoch nicht signifikant, $p = 0,095$. Der qualitative Score steigt zwar numerisch leicht von 0,923 über 0,928 auf 0,933, bleibt aber ebenfalls nicht signifikant, $p = 0,990$.

Ein signifikanter Unterschied zeigt sich nur bei den Tokenmetriken. Der normalisierte Kontexttokenverbrauch sinkt von 154.701 bei Low über 145.658 bei Medium auf 129.540 bei High. Der Friedman-Test ergibt $p = 0,0409$. Auch Tokens pro bestandenen Testfall sinken von 9.468 über 8.763 auf 7.655; der Friedman-Test ergibt $p = 0,0331$. Die paarweisen Post-hoc-Vergleiche bleiben nach Holm-Korrektur jedoch nicht signifikant. Der Effekt ist daher als schwacher globaler Hinweis auf bessere Token-Effizienz durch strukturiertere Prompts zu interpretieren, nicht als stabiler paarweiser Unterschied.

Damit wird Hypothese H3 angenommen. Der Friedman-Test ergibt für die normalisierten Kontexttokens $p = 0,0409$ und für Tokens pro bestandenen Testfall $p = 0,0331$ – beide unter

dem festgelegten Signifikanzniveau von $\alpha = 0,05$. Der numerische Rückgang von 154.701 über 145.658 auf 129.540 Kontexttokens stützt diesen Befund konsistent. Die paarweisen Post-hoc-Vergleiche bleiben nach Holm-Korrektur zwar nicht signifikant, was die globale Effektgröße als moderat einordnet; der globale Friedman-Befund ist für die Annahme der Hypothese jedoch maßgeblich. Über die Hypothese hinaus zeigt sich, dass höhere Prompt-Qualität weder die Hidden-Test-Robustheit ($p = 0,695$) noch die Constraint-/Hallucination-Compliance ($p = 0,558$), den qualitativen Score ($p = 0,990$) oder die Codekompaktheit (SLOC $p = 0,095$) signifikant beeinflusst. Diese Effekte sind jedoch nicht Gegenstand von H3, sondern werden separat unter H4 betrachtet.

6.7 Ökonomische Token-Effizienz und abnehmender Grenznutzen

Die erste Hypothese H1 postulierte einen Sättigungseffekt des Tokenverbrauchs: zusätzlicher Tokeneinsatz sollte im untersuchten Aufgabenraum keine signifikant höhere funktionale Leistung mehr erklären. Die vierte Hypothese H4 nahm an, dass Expert-Prompts neben dem reduzierten Tokenverbrauch auch einen signifikant höheren qualitativen Gesamtscore erzeugen. Zur Prüfung dieser Annahmen wurden die normalisierten Kontexttokens, Tokens pro bestandenen Testfall sowie die logarithmische Regressionsanalyse des Tokenverbrauchs ausgewertet.

Die Regressionsanalyse zeigt keinen belastbaren positiven Zusammenhang zwischen höherem Tokenverbrauch und Test-Pass-Rate. Im OLS-Modell zur Test-Pass-Rate beträgt der Koeffizient für `log_context_tokens_normalized` $-0,00037$ bei $p = 0,913$. Das Modell erklärt mit $R^2 = 0,0105$ praktisch keine Varianz der Test-Pass-Rate. Zusätzliche Tokens kaufen in diesem Datensatz also keine messbar bessere funktionale Leistung.

Regressionskomponente	Koeffizient	p-Wert	Interpretation
<code>log_context_tokens_normalized</code>	$-0,00037$	0,913	kein signifikanter Token-Effekt
<code>architecture_mode = self_refining</code>	0,00174	0,319	kein signifikanter Self-Refining-Effekt
<code>model_name = codex</code>	$-0,00257$	0,374	kein signifikanter Unterschied zur Referenz
<code>model_name = gemini</code>	0,00027	0,935	kein signifikanter Unterschied zur Referenz
Gesamtmodell	—	—	$R^2 = 0,0105$

Damit ist keine sinnvolle Schwelle bestimmbar, ab der zusätzliche Tokens „nicht mehr“ helfen. Stattdessen ist der Aufgabenraum bereits weitgehend im Sättigungsbereich: Die

funktionale Testleistung ist so hoch, dass zusätzlicher Tokenverbrauch kaum noch erklärende Kraft besitzt.

Die ökonomische Betrachtung zeigt besonders deutlich, dass hohe funktionale Leistung mit sehr unterschiedlichem Ressourceneinsatz erreicht wird. In der fairen Artefaktbasis erzielt Codex mit durchschnittlich 62.801 normalisierten Kontexttokens und 3.870 Tokens pro bestandenem Testfall die beste Effizienz. Claude benötigt 112.121 Kontexttokens und 6.998 Tokens pro bestandenem Testfall. Gemini benötigt mit 256.319 Kontexttokens und 14.978 Tokens pro bestandenem Testfall mit Abstand die meisten Ressourcen.

Model l	Test Pass Rate	Ø Kontexttokens	Ø Tokens pro bestandenem Testfall	Ø qualitativer Score	Effizienzbewertung
Claude	100,0 %	112.121	6.998	0,972	hohe Qualität, mittlerer Verbrauch
Codex	99,7 %	62.801	3.870	0,942	beste Token-Effizienz
Gemini	100,0 %	256.319	14.978	0,881	funktional stark, aber ineffizient

Hypothese H1 wird damit angenommen. Sie war als Sättigungshypothese formuliert: zusätzlicher Tokeneinsatz sollte die Test-Pass-Rate nicht mehr signifikant erklären. Genau dieses Muster zeigt sich in den Daten. Der Koeffizient für $\log(\text{context_tokens_normalized})$ ist mit $-0,00037$ nicht signifikant von null verschieden ($p = 0,913$), und das Bestimmtheitsmaß $R^2 = 0,0105$ belegt, dass der Tokenverbrauch praktisch keine Varianz der Test-Pass-Rate erklärt. Der untersuchte Aufgabenraum befindet sich somit erkennbar im Sättigungsbereich.

Hypothese H4 wird nicht bestätigt. High-Level-Prompts verbessern zwar numerisch die Token-Effizienz, erzeugen aber keinen signifikanten Qualitätsgewinn. Der qualitative Score steigt von Low über Medium zu High nur geringfügig und nicht signifikant. Auch Constraint-/Hallucination-Compliance und Hidden-Test-Robustheit verbessern sich nicht signifikant. Expert-Prompts verschieben die Effizienzkurve daher nicht belastbar nach oben. Sie können in einzelnen Konfigurationen den Ressourcenverbrauch senken, erzeugen aber keinen statistisch gesicherten Anstieg des qualitativen Gesamtscores.

Aus betrieblicher Perspektive ist daher nicht das teuerste oder tokenintensivste Modell die beste Wahl, sondern das Modell mit dem besten Verhältnis aus funktionaler Leistung und Ressourcenverbrauch. In dieser Untersuchung ist das Codex. Gemini erreicht zwar perfekte funktionale Werte, benötigt dafür aber ein Vielfaches der Tokens. Claude erreicht in regulär ausführbaren Läufen die höchste Artefaktqualität, ist jedoch operativ nicht über alle 96 Konfigurationen stabil.

In dieser Untersuchung erfüllt Codex dieses Kriterium am deutlichsten.

6.8 Fazit und Synthese der Ergebnisse

Die Untersuchung zeigt zwei unterschiedliche, aber komplementäre Befunde. In der operativen Vollauswertung über alle 96 Konfigurationen sind Codex und Gemini operativ stabiler als Claude, weil Claude 26 Konfigurationen technisch nicht regulär abschließt. Dieser Befund ist für den praktischen Einsatz relevant, da ein Agentensystem den vollständigen Aufgabenraum stabil bearbeiten muss.

In der fairen $n = 70$ -Teilstichprobe verschwindet dieser funktionale Unterschied jedoch. Claude, Codex und Gemini sind anhand von Success Rate und Test Pass Rate nicht signifikant unterscheidbar. Genau dadurch bestätigt sich die Ausgangsthese der Arbeit: Reine Unit-Test-Metriken verlieren bei leistungsstarken Coding-Agenten an Trennschärfe.

Der eigentliche Erkenntnisgewinn entsteht erst durch die multidimensionale Artefaktanalyse. Die zusätzlichen Metriken zeigen signifikante Unterschiede in Test-Robustheit, Codeumfang, zyklomatischer Komplexität, Minimalität, Constraint-/Hallucination-Compliance, Laufzeit und Token-Effizienz. Claude zeigt in regulär ausführbaren Läufen die höchste Test-Robustheit und den höchsten qualitativen Gesamtscore. Codex zeigt die beste Token-Effizienz und die niedrigste zyklomatische Komplexität. Gemini ist funktional stabil, aber deutlich ressourcenintensiver und schwächer bei Constraint-/Hallucination-Compliance.

Die vier Hypothesen wurden über die in Kapitel 4 dargestellten Verfahren geprüft. Jede Hypothese wird primär anhand des jeweils a priori festgelegten globalen Tests bewertet; ergänzende Kennzahlen werden zur inhaltlichen Einordnung berichtet. Paarweise Post-hoc-Vergleiche werden Holm-korrigiert interpretiert. Wo innerhalb einer Hypothese mehrere paarweise Post-hoc-Vergleiche durchgeführt wurden – wie in den Abschnitten 6.4 und 6.6 – sind diese p-Werte explizit Holm-korrigiert. Die globalen Tests werden gegen $\alpha = 0,05$ entschieden. Zwei der vier Hypothesen können auf Grundlage der vorliegenden Daten angenommen werden, zwei werden abgelehnt:

Hypothese	Inhalt	Ergebnis	Begründung
H1	Im untersuchten Aufgabenraum besteht ein Sättigungseffekt: zusätzlicher Tokeneinsatz erklärt keine signifikant höhere Test-Pass-Rate.	angenommen	OLS-Koeffizient für $\log(\text{context_tokens_normalized}) = -0,00037$ ($p = 0,913$); $R^2 = 0,0105$. Der Tokenverbrauch erklärt praktisch keine Varianz der Test-Pass-Rate.
H2	Self-Refining verbessert die Hidden-Pass-Rate signifikant gegenüber direkter Generierung.	abgelehnt	Wilcoxon-Signed-Rank-Test der Hidden-Pass-Rate $p = 0,143$ (Vollauswertung) bzw. $p = 0,179$ (faire Artefaktbasis); Test-Pass-Rate $p = 0,317$; Tokenverbrauch $p = 0,157$.

			Globale Tests ohne Holm-Korrektur, da pro Hypothese ein einzelner Vergleich.
H3	Höhere Prompt-Qualität senkt den normalisierten Tokenverbrauch signifikant.	angenommen	Friedman über drei Prompt-Level: Kontexttokens $p = 0,0409$ (154.701 → 145.658 → 129.540); Tokens pro bestandem Testfall $p = 0,0331$ (9.468 → 8.763 → 7.655).
H4	Expert-Prompts erhöhen den qualitativen Gesamtscore signifikant bei gleichzeitig reduziertem Tokenverbrauch.	abgelehnt	Friedman über qualitativen Score $p = 0,990$ (0,923 → 0,928 → 0,933). Da bereits der globale Test nicht signifikant ist, entfällt eine Post-hoc-Analyse.

Die angenommenen Hypothesen H1 und H3 ergeben zusammen ein konsistentes Bild: Im untersuchten Aufgabenraum kauft zusätzlicher Tokenverbrauch keine bessere Funktionalität (H1), während strukturierte Prompts den Tokenverbrauch je Lauf signifikant senken (H3). Die abgelehnten Hypothesen H2 und H4 zeigen, dass weder zusätzliche Selbstkorrekturschleifen noch ausschließlich höhere Prompt-Qualität einen statistisch belastbaren Qualitätsgewinn erzeugen. Der Direct-Modus ist gegenüber Self-Refining ausreichend, weil Self-Refining keinen signifikanten Robustheitsgewinn erzeugt. High-Level-Prompts können als strukturierende Maßnahme sinnvoll sein, sind aber in dieser Untersuchung nicht als signifikanter Qualitätshebel nachweisbar.

Die höchste Gesamteffizienz entsteht daher nicht pauschal durch das teuerste Modell oder die komplexeste Agentenarchitektur, sondern durch eine ressourcenbewusste Kombination aus effizientem Modell, kontrollierter Prompt-Struktur und Verzicht auf unnötige Reparaturerschleifen. Für den vorliegenden Datensatz ist Codex im Direct-Modus der stärkste betriebliche Effizienz kandidat, während Claude bei regulären Outputs qualitativ stark ist und Gemini vor allem durch operative Stabilität, aber nicht durch Ressourceneffizienz überzeugt.

Damit zeigt die Untersuchung nicht nur, welcher Agent Unit-Tests besteht, sondern wie diese Lösungen entstehen: mit welchem Ressourcenaufwand, welcher Test-Robustheit und welcher Artefaktqualität. Genau diese Differenzierung wäre durch klassische Pass/Fail-Benchmarks nicht sichtbar.

7. Zusammenfassung, Ausblick und Bewertung

7.1 Zusammenfassung

Ziel dieser Arbeit war die Konzeption, Implementierung und statistische Absicherung eines multidimensionalen Evaluations-Frameworks für LLM-basierte Coding-Agenten, das über die reine funktionale Korrektheit hinausgeht. Hierzu wurde eine bestehende Pipeline um statische Code-Metriken (LOC, SLOC, zyklomatische Komplexität, Maintainability Index), AST-gestützte Heuristiken zur Erfassung von Constraint-Verletzungen und Hallucination Flags, zusätzliche Hidden-Tests sowie eine providerübergreifende Token-Normalisierung erweitert. Auf dieser Grundlage wurden drei agentische Coding-CLIs – Claude Code, OpenAI Codex und Gemini CLI – über einen identischen Aufgabenraum aus vier Aufgaben, vier Varianten, drei Prompt-Leveln und zwei Architekturmodi verglichen. Insgesamt wurden 288 Läufe in einem gepaarten Studiendesign ausgewertet.

Das zentrale Ergebnis bestätigt die Ausgangsthese der Arbeit: In der fairen Teilstichprobe ($n = 70$) sind die Agenten anhand von Success Rate und Test-Pass-Rate funktional nicht mehr signifikant unterscheidbar. Die reine Pass/Fail-Bewertung verliert bei leistungsstarken Coding-Agenten ihre Trennschärfe (Ceiling-Effekt). Erst die sekundären Metriken differenzieren die Agenten: Claude erzielt die höchste Test-Robustheit und den höchsten qualitativen Gesamtscore, ist über den vollständigen Aufgabenraum ($n = 96$) jedoch operativ instabil, da 26 Konfigurationen technisch nicht regulär abgeschlossen wurden. Codex erreicht die beste Token-Effizienz und die niedrigste zyklomatische Komplexität. Gemini ist funktional vollständig stabil, benötigt jedoch mit Abstand die meisten Ressourcen und zeigt die schwächste Constraint-/Hallucination-Compliance.

Die vier Hypothesen wurden auf dieser Datenbasis differenziert beantwortet. H1 wird angenommen, weil im untersuchten Aufgabenraum der erwartete Sättigungseffekt eintritt: zusätzlicher Tokeneinsatz erklärt keine signifikant höhere Test-Pass-Rate (OLS $\log(\text{Token})$ $p = 0,913$; $R^2 = 0,0105$). H3 wird ebenfalls angenommen, da höhere Prompt-Qualität den normalisierten Tokenverbrauch über die drei Prompt-Level signifikant senkt (Friedman $p = 0,0409$). H2 und H4 werden hingegen abgelehnt: Self-Refining erzeugt keinen signifikanten Robustheitsgewinn über die Hidden-Pass-Rate ($p = 0,143$ bzw. $0,179$), und Expert-Prompts verschieben die Effizienzkurve nicht signifikant nach oben, da der qualitative Gesamtscore über die drei Prompt-Level nicht signifikant ansteigt (Friedman $p = 0,990$). Insgesamt liefert die Untersuchung damit keine pauschale Empfehlung im Sinne von „mehr Tokens“ oder „mehr Self-Refining“, sondern spricht für eine ressourcenbewusste Konfiguration aus effizientem Modell, strukturierter Prompt-Eingabe und Verzicht auf unnötige Reparaturschleifen.

7.2 Ausblick

Aus den Ergebnissen ergeben sich mehrere Anknüpfungspunkte für weiterführende Untersuchungen. Erstens ist der gewählte Aufgabenraum bewusst auf kompakte, algorithmische Standardaufgaben begrenzt. Genau dadurch entsteht der beobachtete Ceiling-Effekt, der eine Prüfung des in H1 implizit angenommenen anfänglichen Qualitätsanstiegs vor dem Sättigungsbereich verhindert. Künftige Arbeiten sollten daher komplexere, mehrdateiige oder schwächer spezifizierte Aufgaben einbeziehen, um den Bereich abzubilden, in dem zusätzliche Tokens tatsächlich noch funktionalen Mehrwert erzeugen.

Zweitens wurde pro Konfiguration nur ein einziger Lauf erhoben. Da LLMs stochastisch antworten, sollten Wiederholungsmessungen ergänzt werden, um die Streuung innerhalb identischer Konfigurationen zu quantifizieren und Konfidenzintervalle berichten zu können. Drittens ist das im Framework bereits vorgesehene Feld für eine manuelle Qualitätsbewertung bislang nicht befüllt. Eine ergänzende menschliche Code-Review würde es erlauben, die automatisierten Heuristiken zu validieren und semantische Qualitätsaspekte zu erfassen, die statische Metriken nicht abbilden.

Viertens beschränkt sich die ökonomische Betrachtung auf eine normalisierte Token-Kennzahl, die ausdrücklich nicht mit den abrechnungsrelevanten Kosten gleichzusetzen ist. Eine Integration belastbarer, modellübergreifender Preisdaten würde eine echte monetäre Kosten-Nutzen- bzw. ROI-Analyse ermöglichen. Schließlich bietet sich eine Ausweitung auf weitere Programmiersprachen, neuere Modellversionen sowie eine gezielte Ursachenanalyse der nicht regulär abgeschlossenen Claude-Läufe an, um operative Instabilität von tatsächlicher Modellleistung sauber zu trennen.

7.3 Kritische Bewertung

Methodisch zeichnet sich die Untersuchung durch ein gepaartes, blockweises Studiendesign aus, in dem alle Agenten exakt dieselben Konfigurationen bearbeiten. In Kombination mit nichtparametrischen Verfahren und einer Holm-Korrektur der paarweisen Vergleiche erlaubt dies eine statistisch kontrollierte Auswertung. Als besondere Stärken sind die providerübergreifende Token-Normalisierung, die saubere Trennung zwischen operativer Stabilität ($n = 96$) und Artefaktqualität ($n = 70$) sowie die durchgängig auditierbaren Einzelläufe hervorzuheben.

Gleichzeitig sind mehrere Einschränkungen zu benennen. Die externe Validität ist durch die geringe Zahl und den begrenzten Schwierigkeitsgrad der Aufgaben limitiert; der Ceiling-Effekt ist unmittelbar Folge dieser Auswahl. Die Hallucination Flags sind regelbasierte Heuristiken und keine echte Halluzinationsmessung; die Sprachkonsistenz-Variable entsprach in den Daten exakt der Hidden-Pass-Information und

wurde daher nicht als eigenständige Metrik interpretiert. Zudem ist der Vergleich streng genommen einer zwischen agentischen CLI-Systemen und nicht zwischen isolierten Basismodellen: Die untersuchten Werkzeuge orchestrieren intern teils mehrere Modelle und unterschiedliche Hilfsschichten. Die Befunde gelten somit auf der Ebene der Agentensysteme, nicht auf der Ebene einzelner Sprachmodelle. Da schließlich keine modellübergreifend vergleichbaren Kostendaten vorliegen, beruhen die ökonomischen Aussagen auf der Token-Effizienz als Näherungsgröße.

In der Gesamtbewertung zeigt die Arbeit dennoch überzeugend, dass eine multidimensionale Evaluation praktisch relevante Unterschiede sichtbar macht, die klassische Pass/Fail-Benchmarks verbergen. Der wesentliche Beitrag ist primär methodischer Natur – das erweiterte Framework und die Normalisierungsmethodik –, während die inhaltlichen Ergebnisse innerhalb des dargelegten Geltungsbereichs valide sind. Für den vorliegenden Aufgabenraum erweist sich Codex im Direct-Modus als der stärkste betriebliche Effizienz kandidat, während Claude bei regulären Outputs die höchste Artefaktqualität liefert und Gemini vor allem durch operative Stabilität, nicht aber durch Ressourceneffizienz überzeugt.

Quellenverzeichnis

Agarwal, V., Pei, Y., Alamir, S. und Liu, X. (2024): CodeMirage: Hallucinations in Code Generated by Large Language Models. arXiv:2408.08333.

Akhtar, M., Reuel, A., Soni, P., Ahuja, S., Ammanamanchi, P. S., Rawal, R., Zouhar, V., Yadav, S., Whitehouse, C., Ki, D., Mickel, J., Choshen, L., Šuppa, M., Batzner, J., Chim, J., Sania, J., Long, Y., Rahmani, H. A., Knight, C., Nan, Y., Raj, J., Fan, Y., Singh, S., Sahoo, S., Habba, E., Gohar, U., Pawar, S., Scholz, R., Subramonian, A., Ni, J., Kochenderfer, M., Koyejo, S., Sachan, M., Biderman, S., Talat, Z., Ghosh, A. und Solaiman, I. (2026): When AI Benchmarks Plateau: A Systematic Study of Benchmark Saturation. arXiv:2602.16763.

Anthropic (2026a): Models overview. Verfügbar unter: Claude API Docs.

Anthropic (2026b): Prompt caching. Verfügbar unter: Claude API Docs.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I. und Zaremba, W. (2021): Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Cochran, W. G. (1950): The Comparison of Percentages in Matched Samples. *Biometrika*, 37(3/4), S. 256–266.

Eichstaedt, K. E., Kovatch, K. und Maroof, D. A. (2013): A less conservative method to adjust for familywise error rate in neuropsychological research: The Holm's sequential Bonferroni procedure. *NeuroRehabilitation*, 32(3), S. 693–696.

Fagerland, M. W., Lydersen, S. und Laake, P. (2013): The McNemar test for binary matched-pairs data: mid-p and asymptotic are better than exact conditional. *BMC Medical Research Methodology*, 13, Artikel 91.

Friedman, M. (1937): The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance. *Journal of the American Statistical Association*, 32(200), S. 675–701.

Google (2026a): Understand and count tokens. Verfügbar unter: Google AI for Developers, Gemini API Docs.

Google (2026b): Gemini API reference. Verfügbar unter: Google AI for Developers.

Google (2026c): Context caching. Verfügbar unter: Google AI for Developers, Gemini API Docs.

Halstead, M. H. (1977): Elements of Software Science. Operating, and Programming Systems Series, Band 7. Amsterdam: Elsevier North-Holland.

Holm, S. (1979): A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics, 6(2), S. 65–70.

IBM (2026): Cyclomatic complexity. IBM Documentation. Verfügbar unter: IBM Docs.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J. und Amodei, D. (2020): Scaling Laws for Neural Language Models. arXiv:2001.08361.

Liu, J., Xia, C. S., Wang, Y. und Zhang, L. (2023): Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2305.01210.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A. und Clark, P. (2023): Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.

McNemar, Q. (1947): Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), S. 153–157.

OpenAI (2022): Introducing ChatGPT. Verfügbar unter: OpenAI.

OpenAI (2025): Introducing OpenAI o3 and o4-mini. Verfügbar unter: OpenAI.

OpenAI (2026a): Codex. Verfügbar unter: OpenAI Developers.

OpenAI (2026b): Prompt caching. Verfügbar unter: OpenAI API Documentation.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askeel, A., Welinder, P., Christiano, P., Leike, J. und Lowe, R. (2022): Training language models to follow instructions with human feedback. In: Advances in Neural Information Processing Systems 35 (NeurIPS 2022).

pytest-json-report (2024): pytest-json-report. Verfügbar unter: Python Package Index.

Python Software Foundation (2026): ast — Abstract syntax trees. Verfügbar unter: Python-Dokumentation.

Radon (2020): Welcome to Radon's documentation! Radon 4.1.0 documentation. Verfügbar unter: Radon Read the Docs.

Radon (2024): Radon documentation: Introduction to Code Metrics. Verfügbar unter: Radon Read the Docs.

Romano, J., Kromrey, J. D., Coraggio, J. und Skowronek, J. (2006): Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? Vortrag auf dem Annual Meeting of the Florida Association of Institutional Research.

Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S. und Chadha, A. (2025): A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. arXiv:2402.07927.

SciPy Developers (2026a): `scipy.stats.friedmanchisquare` — Friedman test for repeated samples. SciPy Reference Documentation. Verfügbar unter: SciPy Documentation.

SciPy Developers (2026b): `scipy.stats.wilcoxon` — Wilcoxon signed-rank test. SciPy Reference Documentation. Verfügbar unter: SciPy Documentation.

statsmodels Developers (2026a): `statsmodels.stats.contingency_tables.cochrans_q` — Cochran's Q test. statsmodels Documentation. Verfügbar unter: statsmodels Documentation.

statsmodels Developers (2026b): `statsmodels.stats.contingency_tables.mcnemar` — McNemar test of homogeneity. statsmodels Documentation. Verfügbar unter: statsmodels Documentation.

statsmodels Developers (2026c): `statsmodels.regression.linear_model.OLS` — Ordinary Least Squares. statsmodels Documentation. Verfügbar unter: statsmodels Documentation.

Sun, W., Fang, C., Miao, Y., You, Y., Yuan, M., Chen, Y., Zhang, Q., Guo, A., Chen, X., Liu, Y. und Chen, Z. (2023): Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We? arXiv:2312.00413.

Sun, X., Ståhl, D., Sandahl, K. und Kessler, C. (2025): Quality Assurance of LLM-generated Code: Addressing Non-Functional Quality Characteristics. arXiv:2511.10271.

Tambon, F., Dakhel, A. M., Nikanjam, A., Khomh, F., Desmarais, M. C. und Antoniol, G. (2024): Bugs in Large Language Models Generated Code: An Empirical Study. arXiv:2403.08937.

Wilcoxon, F. (1945): Individual Comparisons by Ranking Methods. Biometrics Bulletin, 1(6), S. 80–83.